Contents

1. A First Look at C **5**
1.1 A complete program . . . 5
1.2 Variables and arithmetic . . . 5
1.3 Characters and the input stream . . . 6
1.4 An array and a simple histogram . . . 6
1.5 Functions as named operations . . . 6
1.6 Counting words: states rather than lines . . . 7
Trace a program . . . 7
1.7 When the simplest program is not enough . . . 7
Further exercises . . . 8

2. Types, Expressions, and Control **8**
2.1 Integer types and ranges . . . 8
2.2 Conversion is an operation . . . 8
2.3 Conditions and loops . . . 8
2.4 Short circuit and sequencing . . . 9
2.5 Floating point . . . 9
2.6 C23 spelling and initialization . . . 9
2.7 Integer conversion in practice . . . 9
2.8 Bitwise operations . . . 9
2.9 Evaluation order is independent of precedence . . . 10
2.10 A checked average . . . 10
Further exercises . . . 10

3. Functions and Program Structure **10**
3.1 Definitions, declarations, and headers . . . 10
3.2 Scope and lifetime . . . 11
3.3 Recursion . . . 11
3.4 Design by contract . . . 11
3.5 Out-parameters and explicit failure . . . 11
3.6 Header contracts and linkage . . . 12
3.7 Recursion versus iteration . . . 12
3.8 Exercises . . . 12

4. Pointers, Arrays, and Strings **13**
4.1 Address and indirection . . . 13
4.2 Array arguments carry no length . . . 13
4.3 Strings are terminated byte sequences . . . 13
4.4 Copying bounded text . . . 13
4.5 Constness and pointer arithmetic . . . 14
4.6 A pointer is not an array descriptor . . . 14
4.7 Two-dimensional arrays . . . 14
4.8 Lifetime is as important as address . . . 15
4.9 Restrict and aliasing . . . 15
4.10 Exercises . . . 15

5. Structures and Data Representation **15**
5.1 A value type . . . 15
5.2 Tagged unions . . . 16
5.3 Linked records . . . 16
5.4 Bit fields and explicit masks . . . 16
5.5 Field order, padding, and ABI . . . 16
5.6 A bounded ring buffer . . . 17
5.7 Opaque structures as a design choice . . . 17
5.8 Exercises . . . 17

6. Input, Output, and Files **18**
6.1 Reading lines safely . . . 18
6.2 Formatted I/O . . . 18
6.3 Binary I/O . . . 18
6.4 Files and cleanup . . . 18
6.5 The difference between text and bytes . . . 19
6.6 A bounded line counter . . . 19
6.7 Reading numeric text . . . 19
6.8 File cleanup paths . . . 20
6.9 Exercises . . . 20

7. Dynamic Memory and Ownership **20**
7.1 Allocation is a request . . . 20
7.2 One owner, one release . . . 20
7.3 Resize without losing the original . . . 21
7.4 Fixed-capacity alternatives . . . 21
7.5 A resource table . . . 21
7.6 A checked growable vector . . . 21
7.7 An arena with an explicit alignment policy . . . 21
7.8 Failure-safe initialization . . . 22
7.9 Exercises . . . 22

8. The Preprocessor and Separate Compilation **22**
8.1 Headers describe interfaces . . . 22
8.2 Prefer typed operations . . . 23
8.3 Conditional compilation . . . 23
8.4 Compile and link . . . 23
8.5 What belongs in an interface? . . . 23
8.6 Macros and evaluation hazards . . . 23
8.7 Build modes and reproducibility . . . 24
8.8 Defensive configuration . . . 24
8.9 Exercises . . . 24

9. The C23 Language **24**
9.1 Booleans and null pointers . . . 24
9.2 Type inference with auto and typeof . . . 24
9.3 Constant expressions . . . 24
9.4 Width-aware integers . . . 25
9.5 Binary literals and digit separators . . . 25
9.6 Attributes . . . 25
9.7 New preprocessor capabilities . . . 25
9.8 Checked arithmetic and bit utilities . . . 25
9.9 A migration policy . . . 25
9.10 Generic selection . . . 25
9.11 Safer use of `typeof` . . . 26
9.12 Checked arithmetic in C23 . . . 26
9.13 Selected portability cautions . . . 26
9.14 Exercises . . . 26

10. Reliable Systems Programming **26**
10.1 Undefined, unspecified, implementation-defined . . . 26
10.2 Check sizes before multiplying . . . 27
10.3 The limits of volatile . . . 27
10.4 Host tests and device tests . . . 27
10.5 Protocol parsing . . . 27
10.6 Review checklist . . . 27

10.7 Establish invariants before optimizing . . . 27
10.8 Bound both time and storage . . . 28
10.9 Concurrency and atomics . . . 28
10.10 Errors are part of the API . . . 28
10.11 Exercises . . . 28
10.12 A bounded packet decoder . . . 28

11. An Extended Example: A Word Index **29**
11.1 Requirements . . . 29
11.2 Implementation . . . 29
11.3 Complexity and extensions . . . 31

11.5 From linear search to a sorted index **31**
11.6 A table with explicit invariants . . . 31
11.7 Complexity, measurement, and design . . . 31
11.8 Additional exercises . . . 31

12. Exercises and Further Reading **32**
A progression of projects . . . 32
How to study source code . . . 32
References . . . 32
A note on editions . . . 32

12.1 Exercises as specifications **32**
12.2 Testing boundaries . . . 33
12.3 Integration project: a streaming index . . . 33
12.4 Integration project: a bounded vector . . . 33
12.5 Reading C with a reviewer . . . 33
12.6 Challenge: a portable protocol reader . . . 33
12.7 Challenge: concurrency . . . 33
12.8 Further reading in practice . . . 33

13. Testing, Diagnostics, and Portability **34**
13.1 Start with the contract . . . 34
13.2 Organize tests by dimensions . . . 34
13.3 Assertions for tests, not input validation . . . 34
13.4 A test harness without external libraries . . . 35
13.5 Warnings and language modes . . . 35
13.6 Sanitizers . . . 35
13.7 Unit tests and property tests . . . 36
13.8 Fuzzing a parser . . . 36
13.9 Compare optimization modes . . . 36
13.10 Run the actual target . . . 36
13.11 Document verified claims . . . 36
13.12 Exercises . . . 37

14. Bits, Bytes, and Portable Data **37**
14.1 Storage units . . . 37
14.2 Masks, shifts, and flags . . . 37
14.3 Endianness is part of the format . . . 38
14.4 Representation and alignment . . . 38
14.5 The meaning of volatile . . . 38
14.6 A complete byte codec . . . 38
14.7 Exercises . . . 39

15. Expressions, Objects, and the Abstract Machine **39**
15.1 Values and objects . . . 39

15.2 Storage duration and scope answer different questions . . . 39
15.3 Sequenced and unsequenced evaluations . . . 40
15.4 Integer overflow and conversions . . . 40
15.5 Effective type and aliasing . . . 40
15.6 Order, lifetime, and optimizers . . . 41
15.7 A small checked interval . . . 41
15.8 Exercises . . . 41

16. Atomics, Threads, and Communication **41**
16.1 A race on a counter . . . 42
16.2 Publication with release and acquire . . . 42
16.3 A bounded single-producer/single-consumer queue . . . 43
16.4 Why index wraparound is not numeric overflow . . . 43
16.5 Lifetime and shutdown . . . 43
16.6 Mutexes remain valuable . . . 43
16.7 C23 and implementation support . . . 43
16.8 Exercises . . . 43

17. Function Pointers, Callbacks, and Interfaces **44**
17.1 A function pointer . . . 44
17.2 A callback with context . . . 44
17.3 A stateful visitor . . . 45
17.4 Sorting and the comparison contract . . . 45
17.5 Function pointers are not object pointers . . . 45
17.6 Interfaces as boundaries . . . 45
17.7 Costs and limits . . . 46
17.8 Exercises . . . 46

18. Text, Encodings, and Robust Parsers **46**
18.1 The byte is not the character . . . 46
18.2 Parse lengths explicitly . . . 46
18.3 Decimal parsing as a checked transition . . . 46
18.4 A bounded UTF-8 decoder . . . 47
18.5 Failure and incomplete input are distinct . . . 47
18.6 The language's execution character set . . . 47
18.7 A secure parsing checklist . . . 47
18.8 Exercises . . . 47

19. Compile-Time Programming and the Preprocessor **48**
19.1 Include files and translation units . . . 48
19.2 Object-like macros . . . 48
19.3 Function-like macros and double evaluation . . . 48
19.4 Stringizing and token pasting . . . 48
19.5 Conditional compilation . . . 48
19.6 C23 embedding and diagnostics . . . 49
19.7 Static assertions . . . 49
19.8 Generic selection . . . 49
19.9 An example of compile-time checks . . . 49
19.10 Exercises . . . 49

Appendix A. A Working Reference to C23 **50**
A.1 Translation model . . . 50
A.2 Fundamental types . . . 50
A.3 Expressions at a glance . . . 50
A.4 Declarations . . . 51
A.5 Storage, linkage, and qualifiers . . . 51
A.6 Control statements . . . 51

A.7 Behavior categories . . . 51
A.8 Selected C23 additions . . . 51
A.9 A disciplined compatibility check . . . 51

Appendix B. The Standard Library by Task **51**
B.1 Length is not capacity . . . 52
B.2 Integer conversion . . . 52
B.3 Byte copy versus overlapping copy . . . 52
B.4 Character conversion . . . 52
B.5 Optionality and freestanding environments . . . 52

Appendix C. Selected Worked Exercises **53**
C.1 Absolute difference without unsigned underflow . . . 53
C.2 Checked parsing of an unsigned decimal . . . 53
C.3 Iterative factorial with an overflow check . . . 53
C.4 Little-endian byte decoding . . . 54
C.5 Cleanup on every path . . . 54

1. A First Look at C

A C program describes operations on values and storage. It can run with very little runtime support, but it gives the programmer correspondingly little protection from mistakes. We begin with programs small enough to understand without an elaborate vocabulary.

1.1 A complete program

```c
#include <stdio.h>

int main(void)
{
    puts("hello, world");
    return 0;
}
```

The `#include` directive makes declarations from a header available. `main` is the function from which a hosted program begins. The braces contain a block. `puts` writes a string followed by a newline. A return value of zero reports success to the execution environment.

Compile with `cc -std=c23 -Wall -Wextra -Wpedantic hello.c -o hello` and run `./hello`. The `cc` command may select a compiler too old for the C23 flag; a modern GCC or Clang may be named explicitly.

The program has a declaration, an expression statement, and a return statement. There is no implicit declaration of `puts`; its prototype comes from `stdio.h`. Unlike historical C, modern C requires function declarations that accurately describe the parameters.

1.2 Variables and arithmetic

```c
#include <stdio.h>

int main(void)
{
    for (int celsius = 0; celsius <= 100; celsius += 20) {
        double fahrenheit = 9.0 * celsius / 5.0 + 32.0;
        printf("%3d %6.1f\n", celsius, fahrenheit);
    }
}
```

The loop creates, tests, and increments `celsius`. In each iteration, `fahrenheit` exists within the loop body. A value like `9.0` is floating point; integer division such as `9 / 5` would discard the fractional part. The `printf` format is an agreement between the programmer and the arguments: `%d` expects an `int`; `%f` expects a floating-point argument (a `double` after promotion).

Types are not interchangeable merely because two values can both be printed as numbers. The compile-time type controls arithmetic, conversions, available operations, and storage requirements.

1.3 Characters and the input stream

```c
#include <stdio.h>

int main(void)
{
    int ch;
    unsigned long lines = 0;

    while ((ch = getchar()) != EOF) {
        if (ch == '\n')
            ++lines;
    }
    printf("%lu\n", lines);
}
```

`getchar` returns an `int`, not a `char`: it must represent every `unsigned char` value *and* the negative sentinel `EOF`. The parentheses in the loop make the assignment happen before the inequality test. The input is read until end-of-file or an input error; a production program checks `ferror(stdin)` to distinguish the two.

1.4 An array and a simple histogram

Suppose a program counts decimal digits in its input:

```c
#include <stdio.h>

int main(void)
{
    unsigned long count[10] = {0};
    int ch;
    while ((ch = getchar()) != EOF) {
        if (ch >= '0' && ch <= '9')
            ++count[ch - '0'];
    }
    for (int i = 0; i < 10; ++i)
        printf("%d: %lu\n", i, count[i]);
    return ferror(stdin) ? 1 : 0;
}
```

A C array is a contiguous sequence of elements. Indexes begin with zero. The array above has exactly ten positions; a value of ten is already out of bounds. Unlike many modern languages, C does not generally check array bounds at runtime.

1.5 Functions as named operations

```c
static int maximum(int a, int b)
{
    return a > b ? a : b;
}
```

A function packages an operation behind a name and an interface. `static` at file scope gives this function internal linkage: other source files cannot refer to it by this name. The conditional operator selects one of two expressions.

Exercises. 1.1 Change the temperature program to print from 100 down to 0. 1.2 Count tabs and spaces separately. 1.3 Count lines longer than a fixed limit without storing whole lines. 1.4 Explain what happens if the histogram test accidentally admits the character `':'`.

1.6 Counting words: states rather than lines

Counting lines required remembering only a number. Counting words requires remembering whether the previous character belonged to a word. A *state* is just the minimum information needed to interpret the next input.

```
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    unsigned long words = 0;
    bool inside = false;
    int ch;

    while ((ch = getchar()) != EOF) {
        bool separator = ch == ' ' || ch == '\n' || ch == '\t';
        if (separator) {
            inside = false;
        } else if (!inside) {
            inside = true;
            ++words;
        }
    }
    if (ferror(stdin)) {
        fputs("input error\n", stderr);
        return 1;
    }
    printf("%lu\n", words);
}
```

The loop recognizes a transition from *outside* to *inside* a word. Notice that the definition of a word depends on the definition of separator. Here only three ASCII whitespace characters count. Changing to `isspace` requires `<ctype.h>` and care to pass only an `unsigned char` value or `EOF`.

A state-machine description is valuable beyond text processing. An embedded serial receiver similarly recognizes transitions among waiting, payload, and checksum states, and can process one byte at a time without allocating a complete input stream.

Trace a program

For input `"  one  two\n"`, write down `inside` and `words` after each character. Two adjacent separators must not count an empty word. End-of-file also must not count an extra word.

1.7 When the simplest program is not enough

The programs in this chapter fit in small files, but each has an implicit contract. A histogram counter might overflow on a sufficiently large input. An I/O operation may fail before reaching EOF. An unexpected input byte may not be meaningful under an intended encoding. A reliable C program deliberately records which of those conditions it will handle.

Three habits will recur throughout the book:

1. Carry buffer sizes explicitly.
2. Treat resource and I/O failures as ordinary control paths.
3. Use the type system and compiler diagnostics to expose assumptions rather than conceal them.

Further exercises

1.5 Modify the word counter to report separately how many words began with an ASCII digit. **1.6** Implement the line counter using a function `count_lines(FILE *stream)` that reports failure separately from a legitimate zero count. **1.7** Determine when `unsigned long` is wide enough for your intended input volume; document a policy for counter overflow. **1.8** Write a program that collapses runs of spaces to one space while preserving newlines.

2. Types, Expressions, and Control

The most important questions about an expression are not only what value it computes but which type it uses and whether every intermediate operation is defined.

2.1 Integer types and ranges

C provides `char`, `short`, `int`, `long`, and `long long`, each with signed and unsigned variants where applicable. The exact widths are implementation-dependent within standard constraints. Use `<stdint.h>` when a protocol requires an exact width such as `uint32_t`; these exact-width typedefs exist only when the implementation provides suitable types. `size_t` from `<stddef.h>` describes object sizes and array indexes.

```
#include <stdint.h>
#include <inttypes.h>
#include <stdio.h>
int main(void)
{
    uint32_t mask = UINT32_C(1) << 7;
    printf("%08" PRIx32 "\n", mask);
}
```

The formatting macros such as `PRIx32` are declared in `<inttypes.h>`, which is included above. Prefer unsigned integers for bit masks and modular arithmetic, not as a generic cure for negative values. Unsigned overflow wraps modulo a power of two; signed overflow has undefined behavior.

2.2 Conversion is an operation

```
int a = 7;
int b = 2;
double x = a / b;         /* 3.0, division occurred as int */
double y = (double)a / b; /* 3.5 */
```

Conversions occur at assignments, calls, returns, and within arithmetic. Integer promotions may change a narrow type to `int`. Mixed signed and unsigned arithmetic deserves particular suspicion: compare values only after choosing an appropriate common domain.

2.3 Conditions and loops

The forms `if`, `else`, `switch`, `while`, `do`, and `for` govern control flow. A controlling scalar expression is false if it compares equal to zero, true otherwise. Use braces even for short branches in code that will evolve.

```
unsigned gcd(unsigned a, unsigned b)
{
    while (b != 0) {
        unsigned remainder = a % b;
        a = b;
        b = remainder;
```

```
    }
    return a;
}
```

Each iteration reduces the second operand unless it is already zero. The termination argument is as useful as the program itself: a loop should have a quantity that progresses toward a bound.

2.4 Short circuit and sequencing

`&&` and `||` evaluate left to right and skip the right operand when its value cannot affect the result. Thus `p != nullptr && *p == 5` can test a pointer before dereferencing it, though `nullptr` requires C23 support. Avoid compact expressions that modify the same scalar multiple times without proper sequencing: `i = i++ + 1` is not a clever increment; it has undefined behavior.

2.5 Floating point

Floating-point values are approximations. A decimal fraction like 0.1 often cannot be represented exactly in binary. Equality comparisons can be appropriate for exact assignments and sentinels; they are often unsuitable as tolerance tests for numerical computations. Relative and absolute tolerances should be chosen from the problem's scale, not copied as universal constants.

2.6 C23 spelling and initialization

C23 recognizes `bool`, `true`, and `false` as keywords, and permits empty initialization:

```
struct point { int x, y; };
struct point origin = {};
bool ready = false;
```

An empty initializer gives the object an empty-initialization value (for this structure, both fields become zero). Check compiler support on older toolchains.

Exercises. 2.1 Implement absolute difference for unsigned integers without underflow. 2.2 Write `gcd` tests including zeros. 2.3 Explain why `int n = sizeof array;` can narrow an object size. 2.4 Determine which operations in a signed integer sum can overflow.

2.7 Integer conversion in practice

Consider `size_t`, an unsigned type used for object sizes. It is tempting to subtract two lengths and then test whether the result is negative:

```
size_t remaining = capacity - used;
```

If `used > capacity`, this subtraction does not produce a negative number; unsigned arithmetic wraps. Instead, establish the precondition before computing the difference:

```
if (used > capacity)
    return false;
size_t remaining = capacity - used;
```

A second trap is evaluating mixed integer types. A comparison involving an unsigned operand may convert a signed value to an unsigned type. It is better to check a negative value before a conversion than to hope the usual arithmetic conversions preserve its mathematical meaning.

2.8 Bitwise operations

Bit masks encode independent boolean properties in one integer. Shift counts must be valid for the promoted left operand's width. For portable protocol data, perform shifts in an explicitly selected unsigned type.

```
#include <stdint.h>
```

```
enum {
    FRAME_READY = 1u << 0,
    FRAME_ERROR = 1u << 1,
    FRAME_RETRY = 1u << 2
};

unsigned flags = FRAME_READY | FRAME_RETRY;
bool retry = (flags & FRAME_RETRY) != 0;
flags &= ~FRAME_RETRY;
```

The `|` operator combines selected bits, `&` tests or retains bits, and `~` complements all bits in the promoted type. Using a mask is not the same as testing a boolean expression with `||` or `&&`; those logical operators produce zero or one and may short-circuit.

An important embedded distinction: masking an integer held in RAM is straightforward; read-modify-write of a device register may trigger hardware-specific behavior. Consult the peripheral's reference manual before applying an apparently innocent `register |= bit`.

2.9 Evaluation order is independent of precedence

Multiplication binds more tightly than addition, but precedence tells the compiler how expressions are grouped, not the order of their operands' evaluation. For example, `f() + g()` does not impose an ordering between calls. The programmer can create a sequence explicitly:

```
int left = f();
int right = g();
int result = left + right;
```

Now the calls happen in the order of the statements, though addition still requires a representable result. Avoid constructions that both modify and read a scalar without the necessary sequencing. Compact code is not an excuse for surprising evaluation.

2.10 A checked average

For integers in a constrained environment, accumulating into a wider type can help, but a wider type is not a proof of no overflow. Even a 64-bit sum can overflow if the input is long enough. A fully safe routine must either check each addition, constrain the input count and element ranges, or use a documented alternative algorithm.

A good interface for an average also distinguishes the empty-input case from a real average of zero. A return value alone does not always provide enough information.

Further exercises

2.5 Write a function that sets and clears one bit of a `uint32_t` given a bit position. Reject positions outside 0–31. **2.6** Show an example in which adding two valid `int` values overflows, and implement a guard that avoids evaluating the addition. **2.7** Explain the difference between `a & b == 0` and `(a & b) == 0`. **2.8** Design a loop over `unsigned` values that handles `UINT_MAX` without wrapping forever.

3. Functions and Program Structure

Good C programs put complexity into small, explicit interfaces. Functions make this possible, but they do not hide ownership, sizes, or error conditions unless the programmer makes those matters part of the contract.

3.1 Definitions, declarations, and headers

```
/* statistics.h */
#ifndef STATISTICS_H
#define STATISTICS_H
```

```c
#include <stddef.h>
double average(const double values[static 1], size_t count);
#endif
```

The `static 1` in an array parameter is a promise that an argument refers to at least one element. It is not a runtime bounds check. For an interface that accepts zero elements, prefer `const double *values` and state whether a null pointer is acceptable when `count == 0`.

```c
#include "statistics.h"

double average(const double *values, size_t count)
{
    if (count == 0) return 0.0; /* choose and document policy */
    double total = 0.0;
    for (size_t i = 0; i < count; ++i) total += values[i];
    return total / (double)count;
}
```

Zero is not always a good error signal. Later we will return status codes and write results through out-parameters.

3.2 Scope and lifetime

A variable defined in a block has block scope. An ordinary local variable generally has automatic storage duration: its lifetime ends when the block invocation ends. A file-scope object has static storage duration. `static` within a function preserves an object's value across calls, but introduces shared state that complicates testing and concurrent execution.

3.3 Recursion

```c
unsigned long factorial(unsigned n)
{
    return n <= 1 ? 1UL : (unsigned long)n * factorial(n - 1);
}
```

This mathematical definition is compact, but unsigned overflow makes its result unsuitable for large inputs, and deep recursion consumes stack. A useful recursive function requires both a terminating case and a bounded resource argument. For most production factorial calculations, an iterative implementation with an overflow check is easier to defend.

3.4 Design by contract

A function that converts a decimal string should say whether leading whitespace is accepted, which characters terminate conversion, how overflow is reported, and whether partial parses are accepted. A good interface often has the form

```c
bool parse_unsigned(const char *text, unsigned *result);
```

A successful call writes the result. A failed call leaves the output unchanged by convention. We can implement this without guessing whether an error sentinel might also be a valid result.

Exercises. 3.1 Implement `parse_unsigned` with overflow detection using `UINT_MAX`. 3.2 Replace a recursive factorial with an iterative, checked version. 3.3 Create a header/source pair and link it with a separate test file.

3.5 Out-parameters and explicit failure

A function that returns an integer can use every integer value as an ordinary result. That leaves no spare sentinel for failure. An interface can instead return success and write the result through a pointer:

```c
#include <stdbool.h>
#include <stddef.h>
```

```c
#include <limits.h>

bool sum_nonnegative(const unsigned *items, size_t n,
                     unsigned *result)
{
    if (result == nullptr || (n != 0 && items == nullptr))
        return false;

    unsigned total = 0;
    for (size_t i = 0; i < n; ++i) {
        if (items[i] > UINT_MAX - total)
            return false;
        total += items[i];
    }
    *result = total;
    return true;
}
```

This function makes two promises: failure leaves `*result` unchanged, and success provides the exact sum. The first promise is maintained by assigning to the output only after all checks succeed. Its caller still has an obligation: the pointer must refer to a writable `unsigned` object. A null check cannot detect a dangling pointer.

The result type `bool` expresses a simple yes-or-no outcome. More elaborate interfaces can return an enumeration of errors: invalid argument, overflow, and unsupported operation are often more helpful than one generic false value.

3.6 Header contracts and linkage

Suppose a program has three translation units, `main.c`, `statistics.c`, and `statistics.h`. The header should state the function's public declaration and any types required to call it. The source file defining the function should include the same header; this makes disagreements visible to the compiler.

An identifier with external linkage may be referred to across translation units. A `static` function at file scope instead has internal linkage. Prefer internal linkage for helpers that are not part of the module's interface. It keeps the global symbol namespace small.

Do not put ordinary external object definitions in headers. If each including source file defines the same external object, linking can fail with multiple definitions. Place a declaration such as `extern unsigned counter;` in the header and exactly one definition `unsigned counter;` in a source file.

3.7 Recursion versus iteration

The mathematical definition of a recursive function does not establish that the implementation fits within the machine's resources. Each recursive invocation may need stack space. A function that explores a linked data structure may have recursion depth proportional to its length.

Use recursion when it reflects a bounded structure clearly. Prefer iteration when stack bounds are critical or when the recursive call can be replaced with an ordinary loop. For embedded programs, provide a maximum-depth argument before treating recursive code as suitable for production.

3.8 Exercises

3.4 Design a `parse_temperature` interface distinguishing invalid text from out-of-range input. **3.5** Implement a checked sum for signed `int` elements. **3.6** Split `sum_nonnegative` into a public header, implementation, and executable tests. **3.7** Show how to preserve the output argument's value after each error. **3.8** Explain the difference between linkage and storage duration.

4. Pointers, Arrays, and Strings

Arrays own contiguous elements; pointers refer to objects or functions. The relationship is close, but the two are not the same thing.

4.1 Address and indirection

```
int value = 12;
int *p = &value;
*p = 13;          /* modifies value */
```

A pointer's validity depends on the referenced object's lifetime, alignment, and access permissions. Dereferencing a null, dangling, or otherwise invalid pointer has undefined behavior. C23 adds the keyword `nullptr` for a null pointer constant; legacy code commonly uses `NULL`.

4.2 Array arguments carry no length

```
#include <stddef.h>

int sum(const int *data, size_t count, long long *out)
{
    if (out == nullptr || (count != 0 && data == nullptr))
        return -1;
    long long total = 0;
    for (size_t i = 0; i < count; ++i)
        total += data[i]; /* caller must ensure representable result */
    *out = total;
    return 0;
}
```

The signature carries a count because a pointer does not record the bounds of the original array. This implementation still assumes the sum fits in `long long`; a fully checked version needs an overflow test before each addition.

4.3 Strings are terminated byte sequences

A C string is an array of `char` with a terminating zero byte. `strlen` measures the number of bytes before that terminator, not necessarily the number of Unicode characters. `sizeof` measures a type or an object, not the current length of a string.

```
char name[] = "C23";
size_t capacity = sizeof name;  /* 4 including '\0' */
```

UTF-8 represents some characters with multiple bytes. A procedure that uppercases arbitrary UTF-8 by manipulating each `char` is not a correct Unicode case converter.

4.4 Copying bounded text

```
#include <stddef.h>

bool copy_text(char *dst, size_t capacity, const char *src)
{
    if (dst == nullptr || src == nullptr || capacity == 0)
        return false;
    size_t i = 0;
    while (i + 1 < capacity && src[i] != '\0') {
        dst[i] = src[i];
        ++i;
```

```
    }
    dst[i] = '\0';
    return src[i] == '\0';
}
```

This function assumes valid source storage and that source and destination do not overlap; it signals truncation rather than silently claiming success. It is not a replacement for a general `memmove`.

4.5 Constness and pointer arithmetic

`const int *p` prohibits modifying an `int` through `p`. `int *const p` prohibits assigning another address to `p`. Pointer arithmetic is defined only within an array object (including the permitted one-past position), subject to the language's constraints. A one-past pointer may be compared or subtracted appropriately but not dereferenced.

Exercises. 4.1 Implement `strlen` using a pointer and compare with an index version. 4.2 Explain the difference between `char *p = "abc";` and `char p[] = "abc";`. 4.3 Implement `reverse` for a mutable byte array of known length. 4.4 Add overlap detection or explicitly reject overlap in a copy API.

4.6 A pointer is not an array descriptor

The expression `sizeof array` is useful inside the scope where `array` is an actual array object. Once an array expression is converted to a pointer for passing to a function, its bound is ordinarily lost.

```
#include <stddef.h>

static size_t count_values(const int *values, size_t count)
{
    size_t found = 0;
    for (size_t i = 0; i < count; ++i)
        if (values[i] != 0) ++found;
    return found;
}

int main(void)
{
    int values[] = {3, 0, 8, 0, 1};
    size_t count = sizeof values / sizeof values[0];
    return count_values(values, count) == 3 ? 0 : 1;
}
```

The caller supplies the element count. The callee accepts that count as a contract; C cannot generally establish that a non-null pointer references as many elements as claimed.

4.7 Two-dimensional arrays

A real two-dimensional array occupies contiguous storage in row-major order:

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

The expression `matrix` in most contexts converts to a pointer to an array of three `int`, not an `int **`. This distinction is essential: a pointer-to-pointer usually points to another pointer object, whereas a 2-D array's rows are laid out directly next to each other.

A function that requires a known column count can state it:

```
int matrix_total(size_t rows, const int values[rows][3])
{
```

```
    int result = 0; /* contract: sum must fit in int */
    for (size_t r = 0; r < rows; ++r)
        for (size_t c = 0; c < 3; ++c)
            result += values[r][c];
    return result;
}
```

C23 retains variable length array support as specified by the standard, with implementation differences relevant to storage-duration choices. Where fixed dimensions suffice, fixed arrays make capacity easier to reason about.

4.8 Lifetime is as important as address

Never return the address of an automatic local object:

```
int *bad_pointer(void)
{
    int local = 7;
    return &local; /* invalid after this invocation ends */
}
```

The problem is not that the stack address must immediately change; it is that the object's lifetime has ended. The behavior of later dereferencing that pointer is not defined. Returning a value, writing through a caller-supplied pointer, or returning an allocated object with a documented owner are legitimate alternatives.

4.9 Restrict and aliasing

The qualifier `restrict` describes an access pattern that permits optimizations when the programmer satisfies its association requirements. It is a promise, not a runtime test. Do not add it to a public API merely because it sounds faster. If two pointer arguments can legally refer to overlapping storage, the interface must not assert a restriction that the caller violates.

4.10 Exercises

4.5 Implement an in-place reversal of a `char` array given its exact length. **4.6** Explain why `int **p = matrix;` is incompatible with a two-dimensional `int matrix[2][3]`. **4.7** Write a function to find a value in a slice represented by pointer and length. **4.8** Return both minimum and maximum through checked out-parameters. **4.9** Explain what happens to pointers into a block after successful `realloc`.

5. Structures and Data Representation

Structures associate fields under one type. Enumerations name integral alternatives. Unions allow members to share storage, but they require disciplined tracking of which interpretation is meaningful.

5.1 A value type

```
struct sample {
    unsigned channel;
    double measurement;
    bool valid;
};

struct sample reading = {
    .channel = 2,
    .measurement = 3.75,
    .valid = true
};
```

Designated initializers name members instead of depending on their order. Structure assignment copies member values. Padding bytes may exist between fields; they are not stable serialized data. Do not write a structure directly to a wire protocol or disk format and assume portability across targets.

5.2 Tagged unions

```
enum value_kind { VALUE_INTEGER, VALUE_REAL };

struct value {
    enum value_kind kind;
    union {
        long integer;
        double real;
    } data;
};
```

The tag establishes which member the surrounding program treats as active. An API should expose constructors and accessors rather than relying on callers to maintain an undocumented convention.

5.3 Linked records

```
struct node {
    int key;
    struct node *next;
};
```

A linked list illustrates the difference between a logical collection and its physical representation: nodes may be discontiguous, each allocation may fail, and traversal follows explicit links. In embedded systems, a bounded array can often be simpler and more predictable than a heap-backed list.

5.4 Bit fields and explicit masks

Bit fields can make local state concise, but details of layout and allocation are implementation-defined. For device registers and portable serialized formats, fixed-width unsigned integers with shifts and masks often make data representation clearer. Hardware access adds concerns about `volatile`, synchronization, barriers, and the device's register semantics; `volatile` alone does not make an operation atomic.

Exercises. 5.1 Represent a timestamp with a structure and validate its fields. 5.2 Implement a fixed-capacity pool of list nodes without `malloc`. 5.3 Encode two unsigned 4-bit fields into a byte and decode them.

5.5 Field order, padding, and ABI

A structure establishes a logical association among members, but not a portable wire representation. A compiler may insert padding to meet alignment requirements. The order of named non-bit-field members is preserved in address order, yet `sizeof(struct record)` may exceed the sum of member sizes.

```
struct record {
    unsigned char kind;
    unsigned count;
};
```

There may be padding between `kind` and `count`, or after the final member. Different target architectures or compiler options may create different representations. Avoid `fwrite(&record, sizeof record, 1, stream)` as a long-term file format unless the representation is intentionally fixed for that exact environment.

To serialize an unsigned multi-byte value, select an exact format and explicitly encode bytes. That code is easier to test and more portable than depending on host endianness or struct layout.

5.6 A bounded ring buffer

A ring buffer holds a fixed number of elements and wraps its index back to the beginning. We use a separate count to distinguish an empty buffer from a full one.

```
#include <stdbool.h>
#include <stddef.h>

enum { QUEUE_CAPACITY = 8 };

struct queue {
    unsigned data[QUEUE_CAPACITY];
    size_t head;
    size_t count;
};

bool queue_push(struct queue *q, unsigned value)
{
    if (q == nullptr || q->count == QUEUE_CAPACITY)
        return false;
    size_t tail = (q->head + q->count) % QUEUE_CAPACITY;
    q->data[tail] = value;
    ++q->count;
    return true;
}

bool queue_pop(struct queue *q, unsigned *out)
{
    if (q == nullptr || out == nullptr || q->count == 0)
        return false;
    *out = q->data[q->head];
    q->head = (q->head + 1) % QUEUE_CAPACITY;
    --q->count;
    return true;
}
```

Initialize the entire queue with `struct queue q = {};`. This implementation is appropriate for sequential calls; it is **not** a concurrent lock-free queue. Sharing it between an interrupt and a thread introduces separate ordering, atomicity, and scheduling requirements.

One invariant states that `count` stays between zero and capacity. Another states that `head` is a valid index. Both remain true after successful and failed operations, assuming a properly initialized queue.

5.7 Opaque structures as a design choice

Sometimes callers need a type's behavior, not its layout. A header may declare `struct handle;` without defining its members. A source file privately defines the structure and exports functions taking pointers to it. This allows implementations to change without recompiling all callers, though allocation and ownership must be carefully arranged.

Opaque types are especially useful for devices, parsers, and protocol sessions. A public, by-value structure is often preferable for tiny value types that need no hidden invariants.

5.8 Exercises

5.4 Trace the queue above across eight pushes, one failed push, three pops, and three more pushes. **5.5** Add a `queue_peek` operation that does not remove an item. **5.6** Test that a failed pop leaves the output unchanged. **5.7** Encode `kind` and `count` in a documented byte order without copying the structure representation. **5.8** Compare the memory requirements of this queue and a linked queue for the same maximum capacity.

6. Input, Output, and Files

C's hosted I/O library is built around streams. A stream may refer to a terminal, ordinary file, or implementation-specific source. Errors should be treated as part of the computation.

6.1 Reading lines safely

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char line[256];
    while (fgets(line, sizeof line, stdin) != nullptr) {
        size_t n = strlen(line);
        bool complete = n > 0 && line[n - 1] == '\n';
        printf("bytes=%zu complete=%s\n", n, complete ? "yes" : "no");
    }
    if (ferror(stdin)) {
        fputs("input error\n", stderr);
        return 1;
    }
}
```

A full buffer does not imply a complete line. The next `fgets` may read the rest. Embedded applications frequently need a bounded protocol parser rather than a line-oriented parser.

6.2 Formatted I/O

`printf` uses a format string to interpret subsequent arguments. A mismatch between conversion specifications and argument types can yield undefined behavior. Never use untrusted input as the format string:

```
printf("%s", user_text); /* fixed format */
```

For integers of exact width, use `<inttypes.h>` formatting macros. For `size_t`, use `%zu`. Avoid relying on the fact that two types happen to share a representation on one platform.

6.3 Binary I/O

```
unsigned char buffer[1024];
size_t got = fread(buffer, 1, sizeof buffer, stdin);
if (got < sizeof buffer && ferror(stdin)) {
    fputs("read failure\n", stderr);
}
```

A binary protocol needs a documented byte order, field widths, bounds, and error policy. Reading bytes into a native structure is not sufficient. The portable approach decodes fields explicitly.

6.4 Files and cleanup

```
FILE *fp = fopen("output.txt", "w");
if (fp == nullptr) {
    perror("output.txt");
    return 1;
}
if (fputs("record\n", fp) == EOF || fclose(fp) == EOF)
    return 1;
```

The short example needs care: if `fputs` fails, short-circuit evaluation skips `fclose`, leaking the open stream. Real cleanup logic must close the file on every path, then report either the write error or the close error. A successful write call does not guarantee that data is durably stored.

Exercises. 6.1 Write a line counter that identifies I/O failure. 6.2 Read a little-endian 32-bit unsigned integer from four bytes. 6.3 Rewrite the file example so it always closes its stream.

6.5 The difference between text and bytes

A text stream may transform representations, notably line endings on some systems. Binary I/O is suited to exact bytes. If a file is a protocol packet or serialized image, open it with `"rb"` or `"wb"` in hosted implementations and define the byte format explicitly.

A read of `n` bytes is not guaranteed to deliver `n` bytes in one call. A short `fread` result must be interpreted using `feof` and `ferror`, and the caller must decide whether a short record is valid or malformed.

6.6 A bounded line counter

Some tasks need only one pass over input and almost no memory:

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

bool count_newlines(FILE *stream, unsigned long *result)
{
    if (stream == nullptr || result == nullptr)
        return false;

    unsigned long count = 0;
    int ch;
    while ((ch = fgetc(stream)) != EOF) {
        if (ch == '\n') {
            if (count == ULONG_MAX)
                return false;
            ++count;
        }
    }
    if (ferror(stream))
        return false;
    *result = count;
    return true;
}
```

The function counts newline characters, which may differ from a human notion of lines if the final input has no trailing newline. Documenting that distinction matters more than choosing one rule universally. The caller owns the stream, so this function neither opens nor closes it.

6.7 Reading numeric text

Using `scanf` indiscriminately can make validation difficult: conversion success, partial input, range errors, and leftover text all matter. For untrusted input, reading a bounded line and then using `strtol` or another checked parser often exposes the contract more clearly.

A parser should specify whether whitespace is accepted, whether leading `+` or `-` is legal, whether trailing characters are rejected, and the range of accepted values. A caller should not infer all of those policies from the function name alone.

6.8 File cleanup paths

A file-writing function commonly needs two independent success checks: whether the write succeeded, and whether closing the stream succeeded. Avoid short-circuiting away the close operation:

```
bool ok = fputs("record\n", fp) != EOF;
if (fclose(fp) == EOF)
    ok = false;
```

This makes the cleanup action unconditional once the stream has been opened successfully. It does not guarantee that written data is durable after a power loss, since operating systems and storage devices may buffer writes.

6.9 Exercises

6.4 Define whether a final non-newline-terminated sequence counts as a line, then implement your policy. **6.5** Decode a four-byte length prefix and reject a payload that would exceed a caller-specified capacity. **6.6** Compare a binary stream with a text stream on a platform that transforms line endings. **6.7** Design error reporting that preserves both a write failure and a close failure. **6.8** Explain why a successful `fwrite` is not a promise of power-failure durability.

7. Dynamic Memory and Ownership

Every object occupies storage for a particular lifetime. Automatic objects usually live for one block invocation. Static objects live for the execution of the program. Dynamic objects, obtained through allocation functions, persist until explicitly freed.

7.1 Allocation is a request

```
#include <stdlib.h>
#include <stdint.h>
#include <stddef.h>

int *make_array(size_t count)
{
    if (count == 0 || count > SIZE_MAX / sizeof(int))
        return nullptr;
    return malloc(count * sizeof(int));
}
```

The multiplication is checked before it can overflow. This interface rejects zero-sized requests. Other interfaces may choose different zero-size semantics, but they should document them. New storage from `malloc` has indeterminate contents. Use `calloc` for zero-initialized storage, while still checking the requested dimensions.

7.2 One owner, one release

```
int *samples = make_array(100);
if (samples == nullptr)
    return 1;
/* use samples[0] through samples[99] */
free(samples);
samples = nullptr;
```

The variable `samples` is not the allocated object; it holds an address. After `free`, all aliases that would access the old object are invalid. Resetting one variable does not repair copies of the address.

A useful API states: who allocates, who frees, whether buffers may be shared, which functions can retain pointers, and which operations invalidate earlier pointers. These are conventions implemented by programmers, not runtime guarantees supplied by the language.

7.3 Resize without losing the original

```
void *tmp = realloc(buffer, new_size);
if (tmp == nullptr) {
    /* buffer remains allocated when new_size > 0 */
    return false;
}
buffer = tmp;
```

The example assumes `new_size > 0` and that `buffer` is a valid reallocatable allocation. Assigning `realloc` directly to the only saved pointer risks a leak on failure. Successful reallocation may move storage, invalidating interior pointers.

7.4 Fixed-capacity alternatives

Embedded systems often trade flexible allocation for predictable resource bounds. A fixed-size arena can allocate successive objects from a caller-provided array. It must honor alignment and report exhaustion; an arena reset invalidates all objects in it. Pools that recycle uniform slots provide a different tradeoff. Neither allocator model is automatically thread-safe.

7.5 A resource table

Resource	Acquisition	Release
Heap block	`malloc`, `calloc`	`free`
File stream	`fopen`	`fclose`
Automatic array	enter block	leave block
Static object	program initialization	program termination

Exercises. 7.1 Implement checked dynamic-array growth. 7.2 Write a fixed-capacity arena with explicit alignment policy. 7.3 Identify double-free, leak, and use-after-free paths in a linked-list teardown.

7.6 A checked growable vector

A dynamic array needs three measurements: the pointer to the allocation, the number of initialized elements, and its capacity. The invariant is `length <= capacity`. Growing is exceptional, not required on every insertion.

```
struct vector {
    int *data;
    size_t length;
    size_t capacity;
};
```

A growth policy might double capacity, but the multiplication needs an overflow check. When capacity cannot double, the allocator must either use a smaller sufficient capacity or report failure. The success of `realloc` invalidates pointers into the old allocation even if the returned address happens to equal the old address; callers should obtain fresh pointers from the vector after a change.

The vector's *ownership* can be stated simply: it owns `data`, its caller owns the vector structure, and exactly one destroy operation releases the allocation. Copying the structure by assignment copies its pointer, not its ownership. Two such copies must not both be destroyed.

7.7 An arena with an explicit alignment policy

An arena hands out storage from a fixed block. Its main benefits are bounded overall consumption and cheap bulk reset. Correct alignment must be established for each allocation. A simple arena supporting only objects with alignment no stricter than `max_align_t` can use storage suitably aligned at its base, round offsets upward without overflow, and reject requests that do not fit.

Alignment is part of the contract, not a detail to ignore because a particular CPU tolerates misaligned loads. DMA buffers and cache maintenance impose further target-specific constraints that a generic C arena does not automatically meet.

7.8 Failure-safe initialization

Consider creating an object that owns a buffer and opens a file. If the second step fails, the first resource must be released. Design initialization so that either every required resource is acquired, or every partial acquisition is undone. A cleanup label can be a sensible C technique when a function has many resource-acquisition steps.

```
int status = 1;
void *buffer = malloc(1024);
if (buffer == nullptr) goto done;
FILE *file = fopen("data.bin", "rb");
if (file == nullptr) goto done;
/* use resources */
if (fclose(file) != 0) goto done;
status = 0;
done:
free(buffer);
return status;
```

This sketch requires `<stdio.h>` and `<stdlib.h>`. It closes the file on the successful path, but it is not a general model for cleanup after all possible failures within the use block. For several resources, initialize handles to known values and centralize cleanup that checks each handle.

7.9 Exercises

7.4 Implement a vector with `init`, `append`, and `destroy`. **7.5** Demonstrate how copying a vector structure creates two pointer aliases. **7.6** Design a resettable arena and describe which pointers become invalid after reset. **7.7** Write a function that acquires two resources and releases both on every failure path.

8. The Preprocessor and Separate Compilation

The C preprocessor transforms translation units before compilation. It helps share declarations and configure programs, but its token-level substitutions must not be mistaken for typed functions.

8.1 Headers describe interfaces

```
/* counter.h */
#ifndef COUNTER_H
#define COUNTER_H

#include <stddef.h>
struct counter { size_t value; };
void counter_increment(struct counter *c);

#endif
```

```
/* counter.c */
#include "counter.h"
void counter_increment(struct counter *c)
{
    if (c != nullptr) ++c->value;
}
```

The header is included by both the definition and callers, allowing the compiler to check their consistency. The include guard prevents repeated definitions within a translation unit.

8.2 Prefer typed operations

```
#define SQUARE(x) ((x) * (x))
```

This macro evaluates its argument twice; `SQUARE(i++)` produces undefined behavior due to unsequenced modifications. A function avoids repeated evaluation:

```
static inline int square(int x) { return x * x; }
```

The function has one parameter evaluation, though signed multiplication can still overflow. `inline` is not a promise that machine code will contain no call.

8.3 Conditional compilation

```
#if defined(DEVICE_TARGET)
    /* device-specific interface */
#else
    /* host implementation */
#endif
```

Keep conditional code small and localized. C23 adds `#elifdef` and `#elifndef` as clearer forms for common conditional cases. Feature-test macros should verify actual support rather than assume the entire standard library is implemented because a compiler accepts `-std=c23`.

8.4 Compile and link

For two files:

```
cc -std=c23 -Wall -Wextra -Wpedantic -c counter.c
cc -std=c23 -Wall -Wextra -Wpedantic -c main.c
cc counter.o main.o -o program
```

The compilation stage separately translates source files; the linker resolves symbols and constructs the executable. An undefined reference usually means the program declared a symbol but no linked object provided its definition.

Exercises. 8.1 Write a minimal `Makefile` for the two-source example. 8.2 Replace a parameterized macro with an inline function. 8.3 Explain the distinction among declaration, tentative definition, and external definition.

8.5 What belongs in an interface?

A header exposes names and types necessary for clients. A private source file keeps the implementation's helper functions, internal objects, and algorithms. Public headers should be self-contained: including one in an otherwise empty translation unit should not require secret prior includes.

A useful test compiles a short file containing only the header and a minimal `main`. This catches hidden dependencies that may otherwise be masked by the include order in a larger build.

8.6 Macros and evaluation hazards

Function-like macros are token substitution, not calls. This matters for evaluation count, operator binding, statement boundaries, and type checking. Compare:

```
#define MIN_BAD(a, b) ((a) < (b) ? (a) : (b))
static inline int min_int(int a, int b)
{
    return a < b ? a : b;
}
```

The macro may evaluate one argument twice, while the function evaluates each call argument once. Even the function does not specify which argument expression is evaluated first; do not write code that depends on call-argument evaluation order.

8.7 Build modes and reproducibility

A program's observed behavior depends on source code, implementation, target, optimization settings, language standard, and linked libraries. Keep warning flags in normal development builds. Use optimization during testing as well as debugging, since undefined behavior may appear differently at different optimization levels.

The C standard does not dictate a build system. A small Makefile can teach separate compilation effectively. Larger projects benefit from explicit dependency tracking, toolchain files, and reproducible CI environments.

8.8 Defensive configuration

Prefer feature tests to compiler-name checks. Detecting a compiler vendor does not prove that a header is available or that a function links with the intended C library. For a C23 library feature, compile and link a minimal program using it before enabling the code path. Document fallbacks for older supported targets.

8.9 Exercises

8.4 Write a header-only compile probe. **8.5** Show why a macro with arguments containing `++` can change program behavior. **8.6** Build a two-file project with a separate `-c` step and explain a linker error. **8.7** Define what toolchain details must be recorded to reproduce a firmware build.

9. The C23 Language

C23 is the 2024 revision of the ISO C standard, commonly called C23. It modernizes familiar C without turning it into C++. Its features are best adopted where they improve clarity or correctness; portable projects must still verify compiler and standard-library implementation support.

9.1 Booleans and null pointers

```c
bool valid = true;
int *ptr = nullptr;
if (!valid || ptr == nullptr) {
    /* handle absence */
}
```

`bool`, `true`, and `false` are language keywords in C23. `nullptr` has type `nullptr_t` and denotes a null pointer constant without relying on a macro such as `NULL`.

9.2 Type inference with auto and typeof

```c
int count = 3;
auto copy = count;            /* copy has type int */
typeof(count) another = 4;    /* another has type int */
```

The C23 `auto` form is a type inference declaration, not the C++ rules in their entirety. `typeof` provides a type based on another expression or type name. C23 also supplies `typeof_unqual`, discarding qualifiers and atomicity from the resulting type in its specified way. These mechanisms can reduce duplicated types but should not obscure API contracts.

9.3 Constant expressions

```c
constexpr int channels = 4;
int measurements[channels];
```

C23 introduces `constexpr` for certain objects initialized by constant expressions. This is not C++ compile-time function evaluation. Keep the distinction between an expression computable at translation time and a runtime `const` object.

9.4 Width-aware integers

Where supported, `_BitInt(N)` and `unsigned _BitInt(N)` provide bit-precise integer types. Their availability and limits depend on implementation support. They can be valuable for numerical kernels and hardware-oriented code, but ordinary public interfaces may prefer portable fixed-width types when widths are already covered by `<stdint.h>`.

9.5 Binary literals and digit separators

```
unsigned mask = 0b1010'0101;
```

Binary literals communicate bit patterns; the single-quote separators make long constants easier to read. A leading `0b` is now standardized rather than a vendor extension.

9.6 Attributes

```
[[nodiscard]] int save_record(void);
[[maybe_unused]] static int debug_setting = 0;
```

Standard attribute syntax provides optional diagnostics and intent. `[[nodiscard]]` encourages checking a result; the exact diagnostic is implementation-dependent. Attributes do not magically enforce a full error-handling policy. C23 also specifies `[[deprecated]]`, `[[fallthrough]]`, and `[[noreturn]]` as well as forms of `[[unsequenced]]` and `[[reproducible]]`.

9.7 New preprocessor capabilities

C23 adds `#embed` for incorporating binary data at translation time and `#warning` for diagnostics. `__VA_OPT__` permits more reliable conditional comma handling in variadic macros. Implementation support for `#embed`, in particular, requires explicit checking.

9.8 Checked arithmetic and bit utilities

C23 specifies `<stdckdint.h>` with `ckd_add`, `ckd_sub`, and `ckd_mul` for checked integer arithmetic, and `<stdbit.h>` for unsigned integer bit operations. These facilities may be missing in otherwise C23-capable runtime libraries. A project that depends on them should run configure-time feature tests. Do not assume a working `-std=c23` flag implies complete header support.

9.9 A migration policy

First build with maximum practical warnings. Remove implicit declarations and invalid conversions; make allocation-size checks explicit; audit dangerous signed arithmetic; identify any reliance on extensions. Introduce `nullptr`, boolean keywords, binary literals, designated/empty initializers, and attributes only when supported by the actual deployment toolchains. A robust migration preserves behavior before improving style.

Exercises. 9.1 Replace a platform-specific `__typeof__` declaration with standardized spelling where available. 9.2 Use `[[nodiscard]]` on a status-returning function and observe diagnostics. 9.3 Write a feature-test program for `<stdckdint.h>`. 9.4 Explain why `constexpr` is not simply another spelling for `const`.

9.10 Generic selection

C's `_Generic` chooses an expression based on a controlling expression's type. It supports type-directed interfaces while remaining a compile-time selection mechanism. Generic selection is not dynamic dispatch.

```
#define magnitude(x) _Generic((x), \
    float: fabsf, \
    double: fabs, \
    long double: fabsl \
)(x)
```

This example requires `<math.h>`. It illustrates a common pattern: select an appropriately typed function, then call it. The supported type list is intentionally explicit; passing an unsupported type is a constraint violation unless a `default` association is provided. `_Generic` predates C23, but remains important when designing a modern C interface.

9.11 Safer use of `typeof`

When an expression's type is cumbersome, `typeof` can keep an interface aligned with the expression itself. It does **not** turn C into a language with automatic lifetime management. A pointer inferred from another pointer has the same potential lifetime hazards.

Use inferred types when duplication creates maintenance risk, but write explicit types when they communicate important width, ownership, or protocol constraints.

9.12 Checked arithmetic in C23

When available, `<stdckdint.h>` supports checked integer calculations that report overflow through a boolean result rather than leaving the programmer to reconstruct each type's boundary conditions.

```c
#include <stdckdint.h>
int result;
if (ckd_add(&result, a, b)) {
    /* the mathematical result is not representable in result's type */
} else {
    /* result contains the sum */
}
```

The above is a C23-library-dependent fragment, not a guarantee that every compiler installing a C23 front end also ships this header. For targets without it, a tested manual check or compiler-specific checked-arithmetic primitive may be appropriate.

9.13 Selected portability cautions

Not all useful C23 additions are equivalent from a portability standpoint. Binary literals and attributes may be available before other features. `#embed` support can vary independently from the core language. Bit-precise integers may have target-specific maximum widths. Headers such as `<stdbit.h>` may be absent from older C libraries.

Keep a portability table stating which implementation combinations were actually tested. The standard defines what a conforming implementation must do; deployment engineering determines which implementation is in hand.

9.14 Exercises

9.5 Build a `_Generic` wrapper that chooses the correct absolute-value function for three floating types. **9.6** Write a small compile probe for `nullptr`, `typeof`, and `constexpr`. **9.7** Compare manual overflow checks with `ckd_add` when available. **9.8** Explain why a compiler's version string alone cannot establish support for `<stdbit.h>`.

10. Reliable Systems Programming

The language gives precise rules for certain operations and deliberately leaves other matters to implementations or gives no meaning to invalid programs. Reliability begins with knowing the boundary.

10.1 Undefined, unspecified, implementation-defined

An **undefined** operation has no required behavior: indexing past an array or signed overflow are examples. **Implementation-defined** behavior obliges an implementation to document a choice, such as properties of `char`. **Unspecified** behavior allows a choice among permitted outcomes without requiring that choice to be documented.

Never make the program's correctness depend on one observed result of undefined behavior, even if a particular build seems stable.

10.2 Check sizes before multiplying

```c
#include <stdint.h>
#include <stdlib.h>
#include <stddef.h>

bool allocate_samples(size_t count, int **out)
{
    if (out == nullptr) return false;
    if (count == 0 || count > SIZE_MAX / sizeof(int)) return false;
    int *p = malloc(count * sizeof *p);
    if (p == nullptr) return false;
    *out = p;
    return true;
}
```

The caller must release `*out` after success. The function leaves `*out` unchanged on failure. For C23, `bool` is a keyword. For maximum compatibility with older dialects, include `<stdbool.h>`.

10.3 The limits of volatile

`volatile` is useful for accesses with externally observable behavior, including certain memory-mapped I/O, when used according to implementation and hardware rules. It is not a synchronization primitive and does not prevent data races. Atomics from `<stdatomic.h>` provide operations with explicit memory-order semantics, when supported by the target's implementation.

10.4 Host tests and device tests

Test numerical functions on a host where instrumentation is available. Enable sanitizers such as AddressSanitizer and UndefinedBehaviorSanitizer where toolchains support them. Unit tests do not replace target validation for memory maps, interrupts, scheduling latency, peripheral behavior, or floating-point differences.

10.5 Protocol parsing

A parser accepting a length and a byte buffer is easier to audit than one that searches indefinitely for a terminator. Distinguish malformed input, insufficient data, overflow, and unsupported versions. Parse into temporary state and publish a complete result only after validation.

10.6 Review checklist

Ask of each interface: Are sizes explicit? Are output objects initialized on success only? Is ownership documented? Are integer conversions checked? Are assumptions about endianness and alignment written down? Can cleanup happen on every error path? Is concurrent access synchronized?

Exercises. 10.1 Enumerate five undefined-behavior risks in a string decoder. 10.2 Design a bounded ring buffer and define its full and empty states. 10.3 Explain why `volatile int flag` is not generally a safe cross-thread signal.

10.7 Establish invariants before optimizing

An invariant is a property required to remain true across operations. A ring buffer may maintain `count <= capacity`; a vector may maintain `length <= capacity`; a parser may maintain `position <= input_length`. The easiest time to discover a broken invariant is at the operation that first violates it.

Checks should precede the risky operation. Checking a pointer after dereferencing it, or checking a sum after signed overflow, is too late. An optimizer may assume undefined behavior never occurs and transform subsequent code accordingly.

10.8 Bound both time and storage

An embedded function should be evaluated not only for logical correctness but also for its worst-case time and memory needs. A loop over a fixed-size buffer has an explicit bound. A dynamically grown table or a scan to an unbounded terminator may not.

Real-time behavior adds scheduling, interrupt latency, contention, and priority inversion. None can be solved merely by choosing C23 syntax. The language supports expressing the operations; the system architecture determines when they can run.

10.9 Concurrency and atomics

Sharing a non-atomic object between threads without appropriate synchronization can create a data race, resulting in undefined behavior. `volatile` does not repair this. Atomics in `<stdatomic.h>` provide carefully specified atomic operations; mutexes and other primitives may be preferable where a critical section covers several related objects.

Memory-order choices require a proof about which values must be visible to which threads. Start with sequentially consistent operations unless the design has a demonstrated reason to use weaker ordering. Interrupt-to-thread communication also depends on the implementation's interrupt model and target hardware.

10.10 Errors are part of the API

A function's result contract should specify what happens when it receives a null pointer, an invalid length, an unrecognized packet, insufficient output storage, or a resource failure. The failure behavior may need to preserve caller state. Document that guarantee and test it.

The absence of a crash does not prove memory safety. A parser can mis-handle a length while still writing inside allocated storage, and a numerical routine can return a plausible but incorrect value after an invalid conversion.

10.11 Exercises

10.4 Write invariants for a fixed-capacity queue and construct tests preserving them. **10.5** Make a parser reject overflow before computing `offset + length`. **10.6** Describe what synchronization an interrupt-driven producer and thread consumer would require. **10.7** Identify whether each of four sample failures is undefined, unspecified, or implementation-defined behavior.

10.12 A bounded packet decoder

A practical parser should never trust an unvalidated input length. The companion program `examples/packet_decode.c` reads a miniature binary packet with magic byte `0xA5`, version byte `1`, a two-byte little-endian payload length, at most 64 payload bytes, and a one-byte XOR checksum. This deliberately simple checksum demonstrates framing but does **not** provide cryptographic integrity or meaningful protection against intentional tampering.

The parser first checks the minimum header and checksum size. Next it validates the magic and version. It constructs the declared payload length from bytes, without casting the input buffer to a structure. Before accessing the payload, it verifies the declared length against both the fixed maximum and the available input bytes. Only after validating the complete packet and checksum does it write the decoded result into the caller's object.

One subtle design choice is to build a local candidate object and assign it to the caller only on success. This gives failure an important contract: the output remains unchanged. The implementation uses the subtraction `size - HEADER_SIZE - 1` only after proving the total size is at least `HEADER_SIZE + 1`, so the subtraction cannot underflow.

The program also exercises an invalid checksum and truncated input. It was compiled and executed locally using GCC 14 in C2x dialect, with `-Wall -Wextra -Wpedantic -Werror` and AddressSanitizer and UndefinedBehaviorSanitizer enabled. This verifies that executable under the tested configuration, not every possible input or toolchain.

Exercises. 10.8 Add a failure reason enum without changing the output-on-failure guarantee. 10.9 Generate exhaustive byte-sequence inputs for packets of length zero through four. 10.10 Replace the XOR checksum with a documented CRC and explain why even a CRC is not a cryptographic authentication code.

11. An Extended Example: A Word Index

A word counter is a small program that demonstrates streaming input, fixed-capacity state, tokenization, searching, and error reporting. We deliberately avoid dynamic allocation so the memory bound is explicit.

11.1 Requirements

Read standard input one byte at a time. Treat ASCII letters and digits as part of words, fold uppercase ASCII to lowercase, store up to 1024 distinct words of at most 31 bytes, and print each word with its occurrence count. Reject excess distinct words rather than corrupting the table.

This is an ASCII-oriented utility, not a general Unicode word segmenter. Its fixed-size table and linear search make worst-case costs visible.

11.2 Implementation

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include <stddef.h>

enum { MAX_WORDS = 1024, MAX_LEN = 31 };

struct entry {
    char word[MAX_LEN + 1];
    unsigned long count;
};

static struct entry entries[MAX_WORDS];
static size_t used;

static bool letter_or_digit(int c)
{
    return (c >= 'a' && c <= 'z') ||
           (c >= 'A' && c <= 'Z') ||
           (c >= '0' && c <= '9');
}

static char lower_ascii(int c)
{
    if (c >= 'A' && c <= 'Z') c += 'a' - 'A';
    return (char)c;
}

static bool add_word(const char *word)
{
    for (size_t i = 0; i < used; ++i) {
```

```
        if (strcmp(entries[i].word, word) == 0) {
            if (entries[i].count == ~0UL) return false;
            ++entries[i].count;
            return true;
        }
    }
    if (used == MAX_WORDS) return false;
    strcpy(entries[used].word, word); /* bounded by tokenizer */
    entries[used].count = 1;
    ++used;
    return true;
}

int main(void)
{
    char word[MAX_LEN + 1];
    size_t len = 0;
    int ch;
    bool too_long = false;
    while ((ch = getchar()) != EOF) {
        if (letter_or_digit(ch)) {
            if (len < MAX_LEN)
                word[len++] = lower_ascii(ch);
            else
                too_long = true;
        } else if (len != 0 || too_long) {
            if (too_long) {
                fputs("word exceeds limit\n", stderr);
                return 1;
            }
            word[len] = '\0';
            if (!add_word(word)) {
                fputs("word table full or counter overflow\n", stderr);
                return 1;
            }
            len = 0;
        }
    }
    if (ferror(stdin)) {
        fputs("input error\n", stderr);
        return 1;
    }
    if (too_long) {
        fputs("word exceeds limit\n", stderr);
        return 1;
    }
    if (len != 0) {
        word[len] = '\0';
        if (!add_word(word)) return 1;
    }
    for (size_t i = 0; i < used; ++i)
        printf("%-31s %lu\n", entries[i].word, entries[i].count);
    return ferror(stdout) ? 1 : 0;
}
```

The total table storage is fixed. The program checks every insertion, rejects too-long words, and refuses counter

wraparound. The final output is in order of first appearance, not lexicographic order.

11.3 Complexity and extensions

Each word may be compared against up to 1024 existing entries, so the lookup time is linear in the number of distinct words. A hash table can improve average lookup, but then needs collision handling, a load-factor policy, and carefully bounded worst-case behavior. Sorting entries before printing provides alphabetical output without changing the tokenizer.

Exercises. 11.1 Sort the final table alphabetically. 11.2 Replace linear lookup with open addressing. 11.3 Count overflowed word lengths rather than aborting. 11.4 Implement Unicode-aware tokenization using a deliberately selected library, documenting its normalization rules.

11.5 From linear search to a sorted index

The first implementation searches entries in order of first appearance. If there are u distinct words, one lookup may require u comparisons. We can instead keep the table sorted and use binary search to locate an existing word or the place to insert a new one.

For a sorted array with bounds `lo` and `hi`, the active range is half-open: `[lo, hi)`. At each iteration, compute `mid = lo + (hi - lo) / 2`; this avoids the potential overflow of `(lo + hi) / 2`. Compare the word at `mid` with the query. If it sorts earlier, the search continues after `mid`; otherwise it continues at or before `mid`. The range narrows on every iteration.

When `lo == hi`, the search ends. The same position is the insertion point for a new word. Shifting entries one position to the right preserves sort order. Lookup costs logarithmic comparisons, but insertion still takes linear time in the number of entries. This is a useful reminder that improving one operation does not automatically improve the entire algorithm.

11.6 A table with explicit invariants

The sorted-array representation has three important invariants: (1) `used` never exceeds `MAX_WORDS`, (2) entries in `[0, used)` are sorted lexicographically, and (3) every stored word contains at most `MAX_LEN` ASCII bytes followed by a zero terminator. Before incrementing a count, test against `ULONG_MAX`. Before inserting, confirm unused capacity. These checks protect both memory bounds and numerical meaning.

The companion complete program, `examples/word_index.c`, demonstrates sorted indexing, ASCII normalization, fixed storage, and failure reporting. It deliberately treats words exceeding the maximum length as an error instead of silently truncating them into potentially ambiguous keys.

11.7 Complexity, measurement, and design

For a text with n input bytes and u unique words, lexical scanning is linear in n. Binary lookup is $O(\log u)$ string comparisons per completed word; moving records for insertion is $O(u)$ in the worst case. Since each string is at most 31 bytes, comparison work has a fixed bound. The static table capacity also provides a fixed memory bound.

A hash table could give expected near-constant lookup, with tradeoffs for collision policy, load factor, hashing cost, and predictability. A balanced tree could avoid moving array entries but would require more bookkeeping. Neither is categorically superior: requirements determine the data structure.

11.8 Additional exercises

11.5 Prove that the binary-search interval strictly shrinks on every iteration. **11.6** Add a configurable maximum word count, without introducing dynamic allocation. **11.7** Implement a frequency-first ordering at print time, preserving alphabetical order on ties. **11.8** Test the empty input, a final word without newline, repeated separators, a full table, and an overlong word. **11.9** Benchmark insertion workloads with vocabulary ordered forward, reverse, and randomly.

12. Exercises and Further Reading

The best way to learn C is to predict a program's behavior before compiling it, then explain any discrepancy. Exercises should include ordinary cases, boundaries, and failure cases.

A progression of projects

Project A — Bounded line reader. Implement a function that fills a caller-supplied buffer, reports bytes stored, distinguishes newline from EOF, and signals truncation. Write tests for zero-capacity buffers, one-character buffers, long lines, embedded zero bytes, and input errors.

Project B — Integer parser. Parse unsigned decimal text into `unsigned` without `atoi`. Reject empty input and nondigits; detect overflow before multiplying by ten or adding the next digit. Show why checking after an arithmetic overflow is too late.

Project C — A fixed pool. Implement allocation and release of equal-sized records from a fixed array. Reject invalid and repeated releases. Describe whether the pool can be used concurrently, and if so how it is synchronized.

Project D — Binary message reader. Decode a header with a version byte, 16-bit little-endian payload length, and checksum. Check all bounds before decoding. Test malformed lengths and truncated buffers.

Project E — Word index. Improve Chapter 11 with a hash table, sorting, and a test driver. Benchmark it using increasing vocabulary sizes and measure memory usage.

How to study source code

Read short functions and annotate their contracts. What objects must exist at entry? What inputs are valid? Which values may overflow? What state changes after success? What state is preserved after failure? Which implementation-defined assumptions are present?

Compiler warnings are evidence to inspect, not text to suppress reflexively. Sanitizers find bugs only on executed paths; code review and carefully constructed tests remain essential.

References

- ISO/IEC 9899:2024, *Programming languages — C* (normative specification).
- cppreference, C language reference, especially its C23 features and library pages.
- GCC C standards implementation status and command-line dialect options.
- Clang C language status and diagnostics documentation.
- The ISO C Secure Coding Rules and CERT C guidance, for additional defensive-programming perspectives.

A note on editions

This is an independent original textbook. Classic C teaching remains valuable for its economy of examples, but 2026 readers also need accurate discussion of defined behavior, lifetime, ownership, toolchain variation, safer interfaces, and C23's additions. Future manuscript revisions should add tested solutions, further real-world worked examples, portable build checks, and an index before a complete book-length edition is declared.

12.1 Exercises as specifications

An exercise is complete only when it states what the program accepts, what it returns, and how failure is reported. Consider a decimal parser: what does it do with an empty string, leading spaces, a plus sign, invalid characters, and a number larger than the destination type? The interesting part of the problem is often deciding these details before coding.

12.2 Testing boundaries

For a bounded collection with capacity N, test insertion at sizes zero, one, N−1, and N, then one additional insertion that must fail without damaging existing records. For signed arithmetic, test the largest and smallest representable values, opposite signs, and zero. For a byte parser, test all shorter-than-required inputs.

12.3 Integration project: a streaming index

Read Chapter 11's sorted word index and complete the following experiments:

1. Validate a file containing zero words and a file ending with an unterminated word.
2. Check that capitalization is folded only for ASCII letters and that digits remain in the word.
3. Generate 1,024 unique words and verify the table fills without overrun.
4. Attempt one more unique word and confirm the operation fails.
5. Compare counts against an independent reference implementation for randomly generated ASCII input.
6. Measure runtime for sorted and reverse-sorted unique vocabularies to investigate insertion cost.
7. Replace the sorted array with a fixed-capacity open-addressed hash table; state the collision policy, load factor, and failure semantics.
8. Describe how a UTF-8 version would need a more sophisticated definition of words, code points, and normalization.

12.4 Integration project: a bounded vector

Start from `examples/vector.c`. Add `reserve`, `clear`, and `get` operations with contracts. Make failed growth preserve the old allocation and data. Test failed allocation by injecting an allocator that rejects selected requests. Discuss why assigning one vector object to another is not a deep copy.

12.5 Reading C with a reviewer

For every function, identify which variables represent length and capacity, which arithmetic might overflow, which objects are borrowed versus owned, whether outputs change on failure, which operations may allocate, and whether concurrent calls can safely share state. A program may be short and still require serious reasoning.

12.6 Challenge: a portable protocol reader

Design a packet decoder accepting `const unsigned char *bytes` and `size_t length`. A packet contains a magic byte, a version, a two-byte little-endian payload length, payload bytes, and an eight-bit checksum. Reject incomplete packets before reading fields; reject lengths inconsistent with the buffer; do not write the output structure until the checksum passes. Provide cases for exact minimum length, maximum supported payload, bad version, invalid checksum, and length overflow.

12.7 Challenge: concurrency

Design a producer-consumer queue for one interrupt-side producer and one thread-side consumer. State why Chapter 5's plain queue is not automatically safe under concurrent access. Choose either an RTOS synchronization primitive with appropriate interrupt semantics or a target-validated atomic design. Document memory-order assumptions and why the chosen operations are legal in both execution contexts.

12.8 Further reading in practice

After solving a problem, read the relevant normative language clause or trusted reference and compare it with the implementation's documentation. A standards-conforming program is not merely one that a compiler accepts; conformance involves the behavior required by the language and library specification.

13. Testing, Diagnostics, and Portability

A C program can compile successfully and still be wrong. Compiler diagnostics report only what a tool can establish from the program and its selected options. Tests sample behavior for chosen inputs. Static analysis may identify suspicious paths. Sanitizers instrument executable operations. None of these alone proves that a C program is correct, but used together they provide an effective development discipline.

This chapter develops a concrete workflow for the small programs in this book.

13.1 Start with the contract

Before building a test, state what success means. Suppose a function computes the sum of a sequence of unsigned integers. Its contract might say:

- The input is a pointer to at least `count` elements, or a null pointer if `count` is zero.
- The result pointer must address writable storage.
- On success, the output is the exact mathematical sum.
- On failure, the output is unchanged.
- Overflow is an error rather than a request for modular arithmetic.

These statements determine useful tests. They also identify cases that a test **cannot** legitimately exercise: passing a dangling pointer or claiming a buffer has more elements than it actually has violates the caller's contract. Running such a case and observing a crash does not constitute a useful portable API result.

Tests should therefore separate *valid edge cases* from intentional demonstrations of undefined behavior.

13.2 Organize tests by dimensions

For a bounded operation, look for boundaries in several dimensions:

Dimension	Representative tests
Count	zero, one, capacity minus one, capacity, capacity plus one
Value	zero, ordinary value, largest representable value
Text	empty, valid, invalid digit, too long, no trailing newline
Storage	sufficient capacity, insufficient capacity, allocation failure
State	empty collection, full collection, wraparound, repeated use
I/O	clean EOF, short record, read error, write error

A useful test does not merely show that a function returns the expected value. It checks all promised postconditions. If failure must preserve an output, initialize it to a recognizable value before invoking the function, then verify it remains unchanged.

13.3 Assertions for tests, not input validation

The `assert` macro in `<assert.h>` checks a condition when assertions are enabled. Failed assertions terminate the program with a diagnostic. Defining `NDEBUG` can disable them.

```
#include <assert.h>
#include <stddef.h>

static int first(const int *values, size_t count)
{
    assert(values != nullptr && count != 0);
    return values[0];
}
```

This is appropriate only if callers are required to satisfy the precondition and an assertion failure is an acceptable development-time response. It is not a substitute for checking user-supplied input in a production interface. A release build may omit the assertion altogether, after which a bad call still attempts the invalid access.

Never place an operation with required side effects only inside `assert`; disabling assertions would also remove that operation.

13.4 A test harness without external libraries

For a small project, a conventional `main` that returns nonzero on failure is enough:

```
#include <stdbool.h>
#include <stdio.h>

static bool is_even(unsigned n)
{
    return n % 2U == 0;
}

int main(void)
{
    if (!is_even(0) || !is_even(8) || is_even(7))
        return 1;
    puts("tests passed");
    return 0;
}
```

The build system can run the executable and use its exit status to determine success. Larger suites should report the particular case that failed, but a simple test program can be both readable and valuable.

A command such as `printf 'a b c' | ./word_count` tests a whole program through its standard input and output. Such integration tests complement direct function tests, which are better at reaching internal boundary cases.

13.5 Warnings and language modes

A practical first compiler invocation is:

```
gcc -std=c23 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Werror -O2 example.c -o example
```

The exact useful flags vary by compiler, codebase, and target. In particular, `-Wconversion` can be noisy in legitimate numeric work, and `-Werror` can cause builds to fail when the compiler introduces new diagnostics. The correct response is to review warnings and make an explicit project policy, not to silence them indiscriminately.

Compile in the actual language mode. A compiler accepting one C23 keyword does not necessarily implement every feature or library facility. When a project supports more than one compiler, build with both. Different diagnostics often reveal different classes of mistakes.

13.6 Sanitizers

On supported host targets, AddressSanitizer can detect many out-of-bounds accesses, use-after-free errors, and other memory defects. UndefinedBehaviorSanitizer can detect selected undefined operations, such as certain signed overflows or invalid shifts. They do not cover every form of undefined behavior, and their effectiveness depends on executing the erroneous path.

A useful build on a supported GCC or Clang environment is:

```
cc -std=c23 -O1 -g -fno-omit-frame-pointer \
   -fsanitize=address,undefined example.c -o example_san
./example_san
```

Sanitizer-enabled executables are typically host-oriented. Their instrumentation, runtime overhead, and dependencies can make them unsuitable for microcontrollers. Test algorithms on a host, then separately verify target-specific assumptions.

Do not interpret one clean sanitizer run as proof of a fully safe program. Test case selection remains decisive.

13.7 Unit tests and property tests

A unit test checks a specific expected value. A *property test* checks a relationship that should hold for many inputs.

For example, consider an in-place reversal of an array. If the operation is correct, reversing twice should restore the original sequence. The property holds for empty arrays, one-element arrays, and arrays with duplicates. A test driver can generate random arrays and check this relationship.

Similarly, if a serializer and parser implement the same format, decoding an encoded valid value should produce an equivalent value. This does not prove either routine matches an external specification: they could share the same mistake. Include independent golden test vectors alongside round-trip tests.

13.8 Fuzzing a parser

A fuzzer supplies many automatically generated byte sequences and observes whether the program crashes, violates a memory safety check, hangs, or triggers an assertion. A parser interface like

```
bool decode_packet(const unsigned char *bytes,
                   size_t length,
                   struct packet *result);
```

is well suited to fuzz testing because its input is a bounded byte sequence. The harness should supply valid storage of the declared size, initialize output state, invoke the parser, and verify that failure preserves the output.

Fuzzing discovers bugs; it does not establish absence of bugs. Use corpus inputs for known valid packets and edge cases, and preserve any newly discovered failing inputs as deterministic regression tests.

13.9 Compare optimization modes

Undefined behavior can cause surprising differences between `-O0` and `-O2`. A compiler is permitted to optimize under the assumption that no undefined operation occurs. It may therefore eliminate checks that seem defensive if the source has already performed an invalid operation.

When debugging a difference between builds, investigate lifetime, aliasing, integer overflow, uninitialized objects, sequencing, and incorrect function declarations. Do not rely on a particular assembly listing as the definition of C semantics.

13.10 Run the actual target

A host test cannot substitute for hardware validation when code depends on interrupt timing, cache coherency, memory-mapped registers, DMA, alignment restrictions, an RTOS, or the implementation's floating-point behavior.

The most effective architecture separates portable algorithms from target-specific adapters. For a firmware project, a packet decoder can be tested exhaustively on the host while the UART driver is validated on the real board. Integration tests must then establish that the adapter passes the right byte sequences and lengths to the decoder.

13.11 Document verified claims

A verification report should record the compiler and version, C library if relevant, command line, target architecture, test suite, and result. State the difference between *committed test code* and *tests actually executed*. If a test was not run, do not write that it passed.

For this manuscript, `make check` invokes a shell-based regression suite, while a GitHub Actions workflow can provide repeatable hosted checks. Both should be run on the precise committed revision before a stable release is declared.

13.12 Exercises

13.1 Add a test for unsigned-sum overflow that verifies the output argument remains unchanged.

13.2 Write a reversibility property test for a bounded array reversal.

13.3 Create a fuzzer harness that invokes the packet decoder on random byte sequences and checks output preservation.

13.4 Compare warning diagnostics on the same example under GCC and Clang. Explain differences without suppressing them globally.

13.5 State which properties of a ring buffer can be established with single-threaded tests and which need concurrency-specific reasoning.

13.6 Design a verification matrix for a program that must run on an x86-64 development host and an ARM Cortex-M microcontroller.

14. Bits, Bytes, and Portable Data

C gives programmers direct control over integer representations and bytes. That power is especially useful for embedded devices, persistent files, and network protocols. But an integer's mathematical value is distinct from its in-memory representation, and a portable program must not confuse the two.

14.1 Storage units

An object occupies one or more bytes. In C, a byte is the size of a `char` and contains `CHAR_BIT` bits; while eight-bit bytes are overwhelmingly common, the language does not require `CHAR_BIT == 8`. A format specified in *octets* requires eight-bit units. A portable implementation must check that its target can represent and address the format as intended.

```
#include <limits.h>
#include <stdio.h>

int main(void)
{
    printf("bits per byte: %d\n", CHAR_BIT);
}
```

Exact-width integer types such as `uint32_t` exist when the implementation offers a type of exactly the specified width without padding bits. Fixed-width names clarify the protocol's mathematical range, but they do not choose byte order.

14.2 Masks, shifts, and flags

Use unsigned values for bit-level work. A left shift with a count greater than or equal to the promoted operand's width has undefined behavior, as does a negative shift count. The shift operators have their own integer-promotion rules; explicitly selecting a wide enough unsigned operand avoids accidental signed arithmetic.

```
#include <stdint.h>

static uint32_t extract_byte(uint32_t value, unsigned position)
{
    return (value >> (position * 8u)) & UINT32_C(0xff);
}
```

This helper requires the precondition `position < 4`. Without it, the shift count can be invalid. A production interface can check the position and return success through a separate status indicator.

A register read-modify-write operation may not have the semantics of an ordinary RAM variable. For instance, writing a one may clear a device status bit. Apply shifts and masks according to the hardware reference manual, not just the C expression's arithmetic meaning.

14.3 Endianness is part of the format

A little-endian 32-bit unsigned field stores its least significant octet first. A big-endian field stores the most significant octet first. For portable decoding, use shifts and masks on bytes rather than reinterpreting an input buffer as a pointer to `uint32_t`.

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

bool decode_u32_be(const unsigned char *src, size_t n, uint32_t *out)
{
    if (src == nullptr || out == nullptr || n < 4)
        return false;
    uint32_t v = ((uint32_t)src[0] << 24)
               | ((uint32_t)src[1] << 16)
               | ((uint32_t)src[2] << 8)
               | (uint32_t)src[3];
    *out = v;
    return true;
}
```

This assumes eight-bit input bytes as specified by the format. If the environment permits wider bytes, a protocol layer should explicitly define how octets are obtained. The output is assigned only after the bounds check.

14.4 Representation and alignment

An address in a byte stream may not meet the alignment required by an integer type. Converting that address and dereferencing a misaligned integer pointer may be undefined. Even on processors that allow unaligned loads, compiler assumptions or peripheral restrictions may differ. Byte-wise decoding avoids this issue.

Structure padding and bit-field layout may differ across implementations. Avoid transmitting raw structures as protocol records. Instead, define field lengths, byte order, permitted ranges, and checksums. Encode fields explicitly and test them against documented vectors.

14.5 The meaning of volatile

A memory-mapped device register may require volatile-qualified access so that the compiler performs observable reads and writes. But `volatile` does not specify the hardware bus transaction width, cache behavior, atomicity, memory ordering across cores, or synchronization with interrupts. Those require target-specific facilities and device documentation.

When two threads share ordinary application state, prefer atomics or appropriate synchronization rather than a volatile flag. The language's concurrency rules and a hardware peripheral's rules are related but not interchangeable.

14.6 A complete byte codec

See `examples/byte_codec.c`, which encodes and decodes a 32-bit value in big-endian octet order and verifies the result. It checks `CHAR_BIT == 8` at translation time and uses unsigned shifts. Its public-style functions take explicit output capacity and return `false` when storage is insufficient.

The tests exercise zero, a mixed-byte value, and the maximum 32-bit value. An actual protocol should also test malformed lengths, endianness against independent known vectors, and behavior when the caller supplies invalid storage.

14.7 Exercises

14.1 Write `decode_u16_le` with an explicit length and output pointer, preserving the output on failure.

14.2 Explain why `*(const uint32_t *)buffer` is not a portable replacement for a byte decoder.

14.3 Implement a 32-bit rotate-left operation without shifting by the type's width. Test shift counts zero, one, 31, and 32 after defining whether counts are reduced modulo 32.

14.4 Write a packet encoder with a documented maximum length, a version field, and a checksum. Prove that it never writes past its capacity.

14.5 Compare a host byte codec with the actual peripheral register interface of a chosen microcontroller. Identify which assumptions belong to C and which belong to the vendor's device specification.

15. Expressions, Objects, and the Abstract Machine

A C program is not defined by the machine instructions observed in one build. It is defined by the language's abstract machine: a model of evaluation, objects, values, and observable behavior. A compiler is free to transform an implementation as long as it preserves the behavior required of a conforming program.

This principle explains both C's performance and its hazards. A short expression can have a precise meaning, an implementation-dependent meaning, or no defined meaning at all.

15.1 Values and objects

An *object* is a region of data storage whose contents can represent values. An *expression* can produce a value, designate an object, or have side effects. Consider:

```
int number = 7;
int *address = &number;
*address = 9;
```

The pointer contains a value that designates `number`. The indirection expression `*address` is an lvalue; the assignment modifies the designated object. The validity of the operation depends on `number` being alive, accessible, and represented by a suitable pointer.

Not every expression is a modifiable lvalue. A constant expression, a nonmodifiable qualified lvalue, and an array expression cannot simply be treated as destinations for assignment.

15.2 Storage duration and scope answer different questions

Scope describes where an identifier may be used in source code. *Storage duration* describes the lifetime of storage associated with an object. A local `static` object has block scope but static storage duration:

```
unsigned next_identifier(void)
{
    static unsigned value = 0;
    return ++value;
}
```

The object persists across calls. The function has a new problem: unsigned wraparound eventually repeats an identifier. It is also not inherently safe for unsynchronized concurrent calls. A compact interface does not guarantee suitable semantics.

An automatic object, by contrast, usually exists only for the relevant invocation. Returning its address does not extend its lifetime.

15.3 Sequenced and unsequenced evaluations

Consider the expression

```
int result = f() + g();
```

The language does not require the call to `f` to finish before the call to `g`, or vice versa. It does require both results before their addition can produce the final result. If the functions only compute independent values, that freedom usually does not matter.

If both calls modify the same non-atomic scalar object, their unsequenced side effects can make the expression undefined. Write separate statements when an ordering is required:

```
int left = f();
int right = g();
int result = left + right;
```

The full expressions are sequenced in source order. Still check that the addition itself is representable for signed types.

The operators `&&`, `||`, and comma (when used as an operator) introduce sequencing properties that differ from ordinary arithmetic operators. A function-call argument list is not an ordering guarantee.

15.4 Integer overflow and conversions

For an unsigned integer type, arithmetic is performed modulo one more than its maximum representable value. Signed arithmetic overflow has undefined behavior. These are not implementation details that can safely be ignored because a processor uses two's complement.

```
#include <limits.h>
#include <stdbool.h>

bool increment_int(int value, int *out)
{
    if (out == nullptr || value == INT_MAX)
        return false;
    *out = value + 1;
    return true;
}
```

The guard precedes the addition. A post-addition check would already be too late.

Conversion introduces another boundary. Narrowing an integer to a smaller unsigned type follows modular rules. Converting an out-of-range value to a signed integer type has implementation-defined behavior or can raise an implementation-defined signal. Prefer range checks before narrowing, particularly when parsing files or messages.

15.5 Effective type and aliasing

Object access rules restrict which lvalue types may be used to read or write stored objects. A programmer cannot generally take the address of a `float`, cast it to `int *`, dereference it, and expect portable reinterpretation. Doing so can violate alignment and effective-type rules as well as make code sensitive to representation.

For object-representation inspection, an appropriately sized array of `unsigned char` and `memcpy` are usually clearer. Representation inspection does not imply that every bit pattern is a valid value of every destination type.

```
#include <string.h>
#include <stdint.h>

uint32_t bits_of_float(float x)
{
    uint32_t bits = 0;
    /* Requires equal sizes, verified for the selected target. */
```

```
    if (sizeof bits == sizeof x)
        memcpy(&bits, &x, sizeof bits);
    return bits;
}
```

This example exposes representation bytes without violating aliasing rules, but the resulting integer's meaning depends on the target's floating-point representation and byte order. It is **not** a portable way to serialize a floating value.

15.6 Order, lifetime, and optimizers

A valid optimizer may assume that undefined behavior does not occur. Consequently, source checks appearing after invalid operations may be removed. The correct style is to validate the condition *before* executing the operation it protects.

For array access, verify the index against a known count before evaluation of `array[index]`. For allocation, verify multiplication before computing required bytes. For pointer use, ensure the object has not been freed or otherwise had its lifetime ended.

15.7 A small checked interval

Suppose a program writes `count` bytes beginning at `offset` into a buffer of `capacity` bytes. The tempting check `offset + count <= capacity` can itself overflow when the quantities use an unsigned type. Rewrite the requirement into conditions that can be checked safely:

```
#include <stdbool.h>
#include <stddef.h>

bool fits(size_t offset, size_t count, size_t capacity)
{
    return offset <= capacity && count <= capacity - offset;
}
```

The `&&` operator short-circuits. Therefore `capacity - offset` is evaluated only when `offset <= capacity`. This both avoids underflow and expresses the intended invariant.

A complete executable version with boundary tests appears in `examples/checked_interval.c`.

15.8 Exercises

15.1 Explain why `i = i++ + 1` does not provide a portable way to increment a scalar.

15.2 Give an example of block scope combined with static storage duration. Describe the consequences for recursive calls.

15.3 Implement a checked multiplication of two `size_t` values, preserving its output argument on failure.

15.4 Design a test matrix for `fits` including zero capacity, exact fits, large offsets, and `SIZE_MAX`.

15.5 Explain why reading an integer directly from an arbitrary byte-buffer address can violate alignment or effective-type rules.

15.6 Distinguish a language's unspecified evaluation order from an implementation-defined integer conversion.

16. Atomics, Threads, and Communication

Two threads executing the same valid sequential code do not necessarily produce a valid concurrent program. In C, unsynchronized conflicting accesses to a non-atomic object can form a data race and yield undefined behavior. The standard atomic library gives us a way to coordinate access, but it does not eliminate the need to design ownership and progress.

16.1 A race on a counter

Suppose two threads each execute `++counter` one thousand times. The operation is a read, computation, and write; it is not necessarily one indivisible action. Without suitable synchronization, concurrent increments of an ordinary `int` are a data race. A result of two thousand is not guaranteed, and the program has undefined behavior rather than merely a risk of losing a few increments.

An atomic object makes individual accesses atomic:

```
#include <stdatomic.h>

atomic_uint counter = 0;

/* Called by either worker: */
void count_event(void)
{
    atomic_fetch_add_explicit(&counter, 1u, memory_order_relaxed);
}
```

Relaxed ordering suffices when all that matters is a single independent count, not the visibility of some other data. It does not order ordinary memory operations in a way that can publish a complex object.

16.2 Publication with release and acquire

Imagine a producer fills a payload and then signals that it is ready. The consumer must not read the payload before the producer's writes have been made visible through synchronization.

```
#include <stdatomic.h>
#include <stdbool.h>

static int payload;
static atomic_bool ready = false;

void publish(int value)
{
    payload = value;
    atomic_store_explicit(&ready, true, memory_order_release);
}

bool try_consume(int *out)
{
    if (out == nullptr)
        return false;
    if (!atomic_load_explicit(&ready, memory_order_acquire))
        return false;
    *out = payload;
    return true;
}
```

This is a **one-shot publication example**. It is valid only under a protocol in which one producer publishes once, after which `payload` is never modified, and consumers do not access it before an acquire load observes `true`. Reusing the flag for repeated publications without an additional state transition and synchronization would be incorrect.

A release store synchronizes with an acquire load that reads from it, establishing a happens-before relationship for earlier writes. The atomic flag does not itself turn every access to `payload` into an atomic access.

16.3 A bounded single-producer/single-consumer queue

For repeated communication we need a real protocol. A single-producer/single-consumer (SPSC) ring buffer can keep separate atomic positions. Only the producer modifies the write position, and only the consumer modifies the read position.

Reserve one slot to distinguish full from empty. With capacity eight, the queue can hold at most seven items. The producer writes a non-atomic data element, then publishes its advanced write position with release semantics. The consumer acquires the write position before reading the element. The consumer releases its advanced read position after finishing; the producer acquires it before reusing the slot.

The example in `examples/spsc_queue.c` implements precisely this one-producer/one-consumer algorithm. It is **not** a multiple-producer or multiple-consumer queue. It also does not directly establish that its operations are safe to invoke from an interrupt handler on every implementation: that is a separate target-specific question involving lock-free guarantees, runtime support, and interrupt semantics.

16.4 Why index wraparound is not numeric overflow

An SPSC ring normally advances an index with `(index + 1) % CAPACITY`. If index values are always less than capacity, and capacity is suitably small, the addition stays in range. The modulo computes a position within the array. It should not be confused with allowing arbitrary integer overflow.

An invariant for the chosen queue is `0 <= read_index < CAPACITY` and `0 <= write_index < CAPACITY`. Equal positions signify empty; advancing the writer to the reader's current position would signify full and must be rejected.

16.5 Lifetime and shutdown

A queue's memory must remain alive until both participants stop accessing it. Atomic operations do not rescue a buffer freed while a worker still runs. A threaded application therefore needs a termination protocol and a join or equivalent completion mechanism.

This often has greater practical importance than micro-optimizing a memory order. A formally correct queue implementation is still unsafe if its containing object is destroyed too early.

16.6 Mutexes remain valuable

Lock-free algorithms are not necessarily faster, simpler, or more predictable. A mutex protects a group of related operations and is often the right design for moderate contention. In an RTOS, priority inheritance and interrupt-safe API restrictions matter; do not assume that a host C mutex can be used inside an interrupt handler.

Atomicity, mutual exclusion, wait-freedom, and real-time boundedness are different properties. Assess them independently.

16.7 C23 and implementation support

The C atomic facilities are specified by the language and standard library, but target availability and lock-free properties depend on the implementation. Check `atomic_is_lock_free` when relevant and verify the supported interrupt and threading model on the actual hardware. Freestanding environments may provide only a subset of hosted facilities.

On common hosted implementations, compiling threaded examples may require additional compiler and linker options. Keep those options separate from programs that do not require threading.

16.8 Exercises

16.1 Explain why `volatile unsigned counter` does not fix a data race.

16.2 Draw the release/acquire relationships for one successful SPSC enqueue and dequeue.

16.3 Extend the SPSC example with a clear shutdown protocol. Explain object lifetime requirements.

16.4 Determine whether a capacity-eight ring that leaves one slot empty can hold eight elements. Provide tests for full and empty transitions.

16.5 Explain why changing two producer threads to call the same SPSC enqueue operation breaks the algorithm.

16.6 Compare an SPSC queue with a mutex-protected queue for latency, correctness, maintainability, and portability.

17. Function Pointers, Callbacks, and Interfaces

An ordinary function call names an operation directly. A callback allows a caller to supply the operation itself. This is useful for sorting, traversing data, handling events, and separating reusable code from application policy.

17.1 A function pointer

Consider a function that compares two integers:

```
static int compare_int(int a, int b)
{
    return (a > b) - (a < b);
}
```

The subtraction produces -1, 0, or 1 without calculating `a - b`, which could overflow. A pointer to such a function can be declared as:

```
int (*comparison)(int, int) = compare_int;
int result = comparison(4, 7);
```

The parentheses around `*comparison` are essential. Without them, the declaration would describe a function returning a pointer rather than a pointer to a function.

17.2 A callback with context

Practical interfaces often accept both a callback and an opaque context pointer. The context lets the caller maintain state without resorting to global variables:

```
#include <stddef.h>
#include <stdbool.h>

typedef bool (*visit_fn)(int value, void *context);

bool visit_all(const int *items, size_t count,
               visit_fn visitor, void *context)
{
    if (visitor == nullptr || (count != 0 && items == nullptr))
        return false;
    for (size_t i = 0; i < count; ++i)
        if (!visitor(items[i], context))
            return false;
    return true;
}
```

The visitor returns `true` to continue and `false` to stop. In this design `false` from the traversal means either an invalid argument or an early stop; a richer status type could distinguish them.

The `void *` context is opaque to the traversal function. The caller is responsible for supplying a pointer to the appropriate object and keeping it alive for the duration of the call. Converting an object pointer to `void *` and back preserves its intended use when the underlying type and lifetime are correct.

17.3 A stateful visitor

```
struct accumulator { long long total; };

static bool accumulate(int value, void *context)
{
    struct accumulator *state = context;
    state->total += value; /* assume the sum is representable */
    return true;
}
```

A callback can change the caller's state without changing the traversal algorithm. In production, this example should either check overflow or document a bound on the count and input values.

Callbacks are usually synchronous in simple C library interfaces, but not every callback API is synchronous. An asynchronous system must state whether it retains the context pointer, which thread invokes the callback, and how cancellation and destruction interact.

17.4 Sorting and the comparison contract

The standard `qsort` function takes a comparator that accepts pointers to two objects and returns a negative, zero, or positive result. A comparator must be consistent; it must not produce contradictory orderings for the same values.

```
static int order_ints(const void *left, const void *right)
{
    int a = *(const int *)left;
    int b = *(const int *)right;
    return (a > b) - (a < b);
}
```

The casts are appropriate when the comparator is called by `qsort` on an array of `int`. A comparator should not simply return `a - b` because signed subtraction may overflow. The function cannot assume that its arguments refer to arbitrary or mismatched types.

17.5 Function pointers are not object pointers

The language distinguishes pointers to functions from pointers to objects. Do not assume a function pointer can be stored portably in `void *`. Some operating systems provide dynamic-loading interfaces with platform-specific conventions, but those conventions are not general ISO C guarantees.

Likewise, do not manufacture a function pointer from an integer address and call it without platform-specific guarantees. Embedded startup code and vector tables sometimes rely on implementation-defined representations; such code belongs behind a documented target interface.

17.6 Interfaces as boundaries

A driver interface might offer methods to read, write, and close a device:

```
#include <stddef.h>
#include <stdbool.h>

struct byte_sink {
    bool (*write)(void *context, const unsigned char *data, size_t size);
    void *context;
};
```

An application can then accept a `struct byte_sink` instead of knowing whether output goes to UART, a file, or a memory buffer. This style permits host tests using a fake sink. Yet it carries obligations: the context must remain valid, operations need failure semantics, and concurrent access must be synchronized if shared.

17.7 Costs and limits

Callbacks add indirection and may limit compiler inlining. That cost is often negligible compared with I/O and other operations, but it can matter in a small real-time inner loop. Measure rather than assume. A callback-based interface also needs careful error handling so that failures are not silently ignored.

17.8 Exercises

17.1 Implement a visitor that counts elements greater than a threshold supplied in its context.

17.2 Use `qsort` to order signed integers. Test `INT_MIN` and `INT_MAX` to show why a subtracting comparator is unsafe.

17.3 Design a byte sink whose write function may accept fewer bytes than requested, and define a complete-write wrapper.

17.4 Explain why casting a function pointer to `void *` is not a portable storage strategy.

17.5 Show how a synchronous callback's context lifetime differs from an asynchronous callback's lifetime.

18. Text, Encodings, and Robust Parsers

Text is often treated as the simplest form of input, but a text interface combines several independent conventions: how bytes are encoded, what counts as a character, where a line ends, and which text represents a valid value. C gives programmers byte-oriented primitives; a program must choose and document its interpretation of those bytes.

18.1 The byte is not the character

A C string is a sequence of `char` values terminated by a zero byte. The library function `strlen` counts units before the terminator. For UTF-8 text, that count is not necessarily the number of Unicode scalar values or user-perceived characters.

UTF-8 encodes a code point in one to four octets. Continuation octets do not stand alone. A decoder must reject overlong sequences, surrogate code points, values beyond U+10FFFF, and incomplete sequences. A robust decoder also must not read past the caller's buffer when the leading octet suggests more input than remains.

18.2 Parse lengths explicitly

An untrusted message need not have a terminating zero byte. A parser can accept a pointer and a length, so no operation depends on searching outside the supplied region.

```
#include <stddef.h>
#include <stdbool.h>

bool ascii_digit(unsigned char c)
{
    return c >= '0' && c <= '9';
}
```

This function classifies one byte without invoking locale-sensitive character classification. For applications that require whitespace recognition under a locale, `isspace` is useful, but its argument must be `EOF` or a value representable as `unsigned char`.

18.3 Decimal parsing as a checked transition

A decimal parser maintains an accumulated number. Each incoming digit requests a transition from `value` to `value * 10 + digit`. For an unsigned maximum `MAX`, that transition is valid only when

[value \leq \left\lfloor\frac{MAX-digit}{10}\right\rfloor.]

The inequality must be tested *before* multiplication. A checked parser can therefore reject overflow without ever evaluating the overflowing expression.

A complete implementation appears in `examples/parse_unsigned.c`. Its contract is intentionally narrow: only nonempty ASCII decimal digits are allowed, and failure leaves the output unchanged. This is different from `strtoul`, which has defined syntax for optional whitespace, signs, and bases and uses `errno` for range errors.

18.4 A bounded UTF-8 decoder

The companion program `examples/utf8_decode.c` decodes a single Unicode scalar value from a counted input buffer. It reports how many octets were consumed on success and does not change either output on failure.

For example, the sequence `E2 82 AC` represents U+20AC (the euro sign). But `C0 80` must be rejected: it is an overlong representation of U+0000. The byte sequences `ED A0 80` and `F4 90 80` must also be rejected because they denote disallowed scalar values.

The example's purpose is decoding, not Unicode normalization, segmentation, case folding, or character rendering. Those operations require additional specifications and, in real-world applications, frequently a carefully selected text library.

18.5 Failure and incomplete input are distinct

A streaming decoder may encounter a prefix of a potentially valid sequence, such as the first two octets of a three-octet UTF-8 character. A one-shot parser can report failure and ask its caller to provide more bytes. A streaming parser often benefits from three results: valid, invalid, and incomplete.

That distinction prevents a network receiver from prematurely rejecting input merely because it arrived in pieces. It also prevents an actual malformed sequence from being treated as indefinitely incomplete.

18.6 The language's execution character set

C programs have implementation-defined character representations for source and execution environments, within the language's requirements. The ordinary literal `'A'` does not universally imply the integer 65. An ASCII-oriented parser must state its ASCII requirement explicitly.

The examples in this chapter use octet values for UTF-8 and the digit range for decimal input. Such assumptions are appropriate when decoding formats defined in ASCII or UTF-8, but they are not claims about every C execution character set.

18.7 A secure parsing checklist

Before parsing an untrusted input, identify its maximum length, byte encoding, permitted grammar, numeric ranges, truncation policy, and error reporting. Validate a complete candidate before writing durable output state. Avoid relying on null terminators unless the interface guarantees them.

Tests should include zero-length input, all valid boundary values, incomplete prefixes, extra trailing bytes, and malformed encodings. Pair handwritten test vectors with fuzzing, and preserve discovered failures as regression tests.

18.8 Exercises

18.1 Implement a parser for unsigned hexadecimal digits with an explicit buffer length and checked overflow.

18.2 Extend the UTF-8 decoder to distinguish incomplete input from malformed input.

18.3 Explain why byte length, Unicode scalar count, and displayed character count can differ.

18.4 Add fuzz tests checking that malformed input never changes the decoder's outputs.

18.5 Design a streaming line protocol that accepts CRLF and LF line endings while bounding the line length.

18.6 Compare a locale-sensitive classification function with a format-specific ASCII byte test.

19. Compile-Time Programming and the Preprocessor

The C preprocessor is a separate, token-oriented facility that runs before ordinary translation. It can select platform-specific code, define constants, construct repetitive expressions, and include headers. Its power makes restraint important: a macro is not a typed function, and conditional compilation should not obscure the program's logic.

19.1 Include files and translation units

A header normally contains declarations and interface-related definitions. A source file includes that header and supplies the corresponding external definitions. Inclusion is conceptually textual: the preprocessor inserts the header's tokens into the translation unit.

```
#ifndef BUFFER_H
#define BUFFER_H
#include <stddef.h>
struct buffer_view { const unsigned char *data; size_t size; };
#endif
```

The guard prevents repeated processing of the same interface within one translation unit. It does not make two incompatible versions of an interface agree or prevent multiple external definitions across translation units.

19.2 Object-like macros

An object-like macro can name a manifest constant:

```
#define BUFFER_CAPACITY 256
```

For a numeric quantity with an appropriate integer type, an enumeration constant or C23 `constexpr` object can be clearer, subject to the required constant-expression context and toolchain support. A macro has no type of its own; it substitutes preprocessing tokens that are interpreted later.

19.3 Function-like macros and double evaluation

```
#define MAXIMUM(a, b) ((a) > (b) ? (a) : (b))
```

This macro can evaluate a chosen argument twice. An invocation involving `i++` is dangerous even if its parentheses are correct. Replacing a macro with a `static inline` function usually improves type checking and evaluation behavior, but an inline function still has to handle overflow and other ordinary C hazards.

19.4 Stringizing and token pasting

The `#` operator can stringify a macro argument, and `##` can paste preprocessing tokens. These mechanisms are helpful for diagnostics and repetitive declarations but can create opaque interfaces.

```
#define STRINGIZE(x) #x
#define CONCAT(a, b) a ## b
```

For example, `STRINGIZE(hello)` produces the string literal `"hello"`. Token-pasting does not create arbitrary source text; its result must be a valid preprocessing token. Expand-once helper macros may be necessary when callers pass macro arguments whose expanded values should be concatenated.

19.5 Conditional compilation

```
#if defined(PLATFORM_DEVICE)
    /* device adaptation */
#elif defined(PLATFORM_HOST)
    /* host adaptation */
#else
```

```
    #error "Choose a supported platform"
#endif
```

C23 also provides `#elifdef` and `#elifndef`. Use feature probes where possible: checking a vendor name is not proof that a particular header or library function is available. Keep platform branches behind narrow adapter interfaces so most application code can be compiled identically everywhere.

19.6 C23 embedding and diagnostics

The C23 `#embed` directive is designed to incorporate resource data into a translation unit. It is particularly interesting for fixed firmware resources such as calibration tables or static model parameters. Actual support differs among compilers, so a production project needs a feature probe and a fallback generation step.

C23 also standardizes `#warning`. Diagnostics emitted this way can be useful when a configuration is supported but discouraged. Unlike `#error`, a warning does not necessarily stop translation.

19.7 Static assertions

Use `static_assert` to make fundamental assumptions visible at translation time:

```
#include <limits.h>
static_assert(CHAR_BIT == 8, "octet storage required");
```

The constraint is appropriate for an octet-specific wire format; it would be unnecessarily restrictive for a general-purpose algorithm that does not depend on byte width.

19.8 Generic selection

The `_Generic` selection mechanism works on types rather than token spelling. It can choose an appropriate typed operation:

```
#include <math.h>
#define abs_floating(x) _Generic((x), \
    float: fabsf, double: fabs, long double: fabsl)(x)
```

The selected function takes the value once, avoiding a common multiple-evaluation problem of expression macros. This example requires a supported floating type. A more complete public API should decide how to diagnose unsupported types and document conversion policy.

19.9 An example of compile-time checks

The companion program `examples/compile_time.c` combines `static_assert`, stringizing, token pasting, a C23 empty initializer, and a small function. Its checks are deliberately modest: their purpose is to show that preprocessing and language-level type checking cooperate but are not identical mechanisms.

19.10 Exercises

19.1 Explain why a function-like macro can evaluate an argument more than once.

19.2 Implement two-level stringizing that expands a macro before turning its value into a string.

19.3 Write a configuration header that rejects simultaneous host and device target selections.

19.4 Write a compile probe for `#embed` and document a fallback for a toolchain that lacks it.

19.5 Define the differences among `#if`, `static_assert`, and a runtime `if` statement.

19.6 Use a `_Generic` selection to choose one of two distinct, typed integer-printing functions.

Appendix A. A Working Reference to C23

This appendix is a navigational guide, not a replacement for ISO/IEC 9899:2024. Implementations differ in optional features and in how far their C libraries have caught up.

A.1 Translation model

A source file is preprocessed and translated as part of a *translation unit*. Header contents are incorporated by inclusion; macro substitution and conditional inclusion precede ordinary compilation. External symbols are resolved when separately compiled translation units are linked. String literals and static objects are initialized according to the rules for their storage duration.

A.2 Fundamental types

- `void` describes absence of a value.
- `bool` is C23's boolean type. `true` and `false` are keywords.
- Character types include `char`, `signed char`, and `unsigned char`. Plain `char` is a distinct type, with implementation-defined signedness.
- Standard signed integer types are `signed char`, `short`, `int`, `long`, and `long long`; each has an unsigned counterpart.
- Floating types include `float`, `double`, and `long double`, with additional optional IEC-related facilities.
- A pointer type refers to an object or function; it does not automatically encode the size or lifetime of the referenced storage.
- Array types contain sequences of elements. Structures contain named members; unions share storage; enumerations provide named integer constants.
- C23 introduces bit-precise integer types written `_BitInt(N)` and `unsigned _BitInt(N)`, subject to implementation limits.

The sizes of fundamental types are not universally fixed. Test the actual target's limits using `<limits.h>`, `<stdint.h>`, and `<float.h>`.

A.3 Expressions at a glance

From highest binding strength to lowest, a practical *simplified* precedence guide:

Level	Operators or forms	Association
Postfix	`[] () . ->` postfix `++ --`	left to right
Unary	prefix `++ --`, `& * + - ! ~`, `sizeof`, `alignof`	right to left
Cast	`(type)`	right to left
Multiplicative	`* / %`	left to right
Additive	`+ -`	left to right
Shift	`<< >>`	left to right
Relational	`< <= > >=`	left to right
Equality	`== !=`	left to right
Bitwise AND	`&`	left to right
Bitwise XOR	^	left to right
Bitwise OR	‘	‘
Logical AND	`&&`	left to right
Logical OR	‘	
Conditional	`?:`	right to left (special grammar rules)
Assignment	‘= += -= *= /= %= «= »= &= ^=	=‘
Comma	`,`	left to right

Precedence does not imply evaluation order. Thus `f() + g()` does not specify which call happens first. The distinction matters whenever expressions change shared state.

A.4 Declarations

Read a declaration outward from the name, respecting parentheses:

```
int *p;                    /* pointer to int */
int a[8];                  /* array of eight int */
int (*fp)(double);         /* pointer to function returning int */
int *make_value(void);     /* function returning pointer to int */
```

A prototype specifies parameter types. In C23, an empty function-parameter list declares a function with no parameters; `(void)` remains explicit and clear. Avoid relying on earlier C conventions for nonprototype declarations.

A.5 Storage, linkage, and qualifiers

Automatic storage normally begins on block entry and ends when execution leaves the block. Static storage lasts for the program execution. Thread storage is associated with a thread where supported. Allocated storage lasts from successful allocation to deallocation.

At file scope, `static` can give an identifier internal linkage; `extern` can declare an externally linked identifier. `const` restricts modification through a qualified lvalue; `volatile` affects observable accesses but does not provide atomic synchronization; `restrict` is a promise about pointer-based accesses and must only be used when its aliasing conditions hold.

A.6 Control statements

`if` chooses a branch; `switch` dispatches among case labels. `for`, `while`, and `do` iterate. `break` exits a loop or switch; `continue` advances to the next loop iteration; `return` exits a function. `goto` transfers control to a label in the same function, but must obey restrictions on entering variably modified scopes and similar rules.

A.7 Behavior categories

- **Defined:** the standard requires specified results.
- **Implementation-defined:** the implementation chooses and documents a permitted result.
- **Unspecified:** the implementation may choose among permitted results.
- **Undefined:** no requirements are imposed; programs must not depend on an observed behavior.

A program may compile without diagnostics and still execute undefined behavior.

A.8 Selected C23 additions

C23 standardizes binary integer literals, digit separators, empty initialization, `nullptr`, `typeof`/`typeof_unqual`, `auto` type inference, `constexpr` objects, bit-precise integers, attributes, `#embed`, `#elifdef`, and `#elifndef`. The library expands with, among other things, checked integer operations and bit utilities. Do not confuse adoption of C23 syntax in a compiler with complete deployment availability of every library feature.

A.9 A disciplined compatibility check

Compile tiny feature probes using the real target compiler and link against the real target C library. Record compiler version, C library version, standard mode, target architecture, and the precise feature probed. Treat a feature as available only after the probe compiles, links, and where possible executes.

Appendix B. The Standard Library by Task

This is a field guide to the most commonly needed hosted C facilities, not a complete list of the standard's clauses.

Need	Header	Key facilities
Input, output, files	`<stdio.h>`	`FILE`, `fopen`, `fclose`, `fread`, `fwrite`, `fgets`, `printf`
Byte and string operations	`<string.h>`	`memcpy`, `memmove`, `memcmp`, `strlen`, `strcmp`
Allocation, conversion	`<stdlib.h>`	`malloc`, `calloc`, `realloc`, `free`, `strtol`, `qsort`
Integer limits	`<limits.h>`	`INT_MAX`, `CHAR_BIT`, etc.
Exact-width integer types	`<stdint.h>`	`uint32_t`, `SIZE_MAX` and related types/macros
Integer formatting	`<inttypes.h>`	`PRIu32`, `SCNx64` etc.
Type and offset	`<stddef.h>`	`size_t`, `ptrdiff_t`, `offsetof`
Character classification	`<ctype.h>`	`isdigit`, `isspace`, `tolower`
Error indications	`<errno.h>`	`errno`, `ERANGE`
Mathematics	`<math.h>`	`sqrt`, `sin`, `fabs`, `isfinite`
Assertions	`<assert.h>`	`assert`
Variadic functions	`<stdarg.h>`	`va_list`, `va_start`, `va_arg`, `va_end`
Atomics	`<stdatomic.h>`	atomic types and operations
Threads	`<threads.h>`	`thrd_t`, mutexes, condition variables where supported
Checked integer arithmetic	`<stdckdint.h>`	`ckd_add`, `ckd_sub`, `ckd_mul` (C23)
Bit utilities	`<stdbit.h>`	leading/trailing zero counts, bit width and rotation-related utilities (C23)

B.1 Length is not capacity

`strlen` finds a terminating zero byte. It does not accept a buffer size, so calling it on a nonterminated array is unsafe. Functions like `snprintf` take output capacity, and their return values should be checked to detect truncation.

```c
char buffer[16];
int needed = snprintf(buffer, sizeof buffer, "%u", 12345U);
if (needed < 0 || (size_t)needed >= sizeof buffer) {
    /* formatting error or truncation */
}
```

B.2 Integer conversion

`strtol` is usually preferable to `atoi` when handling untrusted text. Set `errno = 0` before conversion, inspect the end pointer, and check `ERANGE`. Even after a successful `long` result, verify the destination type's range.

B.3 Byte copy versus overlapping copy

`memcpy` requires source and destination regions not to overlap. `memmove` permits overlap, but both require valid regions of at least the requested size. Neither performs object-bound checks for you.

B.4 Character conversion

The `<ctype.h>` functions accept `EOF` or a value representable as `unsigned char`. Passing a negative signed `char` directly is generally invalid. For byte-oriented classification, cast:

```c
unsigned char byte = (unsigned char)some_char;
if (isdigit(byte)) { /* ... */ }
```

The character functions can be locale-sensitive. They are not general Unicode text processing primitives.

B.5 Optionality and freestanding environments

A freestanding embedded system may implement only a subset of the hosted library. Standard features may also depend on toolchain, linker, and C library combinations. Document whether examples require a hosted environment, and test all target-specific functions directly.

Appendix C. Selected Worked Exercises

These solutions illustrate contracts and tests rather than prescribe one clever implementation.

C.1 Absolute difference without unsigned underflow

```
unsigned difference(unsigned a, unsigned b)
{
    return a >= b ? a - b : b - a;
}
```

Both subtractions occur only in nonnegative order. Test `(0, 0)`, `(1, 0)`, `(0, UINT_MAX)`, and both operand orders.

C.2 Checked parsing of an unsigned decimal

```
#include <stdbool.h>
#include <limits.h>

bool parse_unsigned(const char *text, unsigned *out)
{
    if (text == nullptr || out == nullptr || *text == '\0')
        return false;

    unsigned value = 0;
    for (const unsigned char *p = (const unsigned char *)text;
         *p != '\0'; ++p) {
        if (*p < '0' || *p > '9') return false;
        unsigned digit = (unsigned)(*p - '0');
        if (value > (UINT_MAX - digit) / 10U) return false;
        value = value * 10U + digit;
    }
    *out = value;
    return true;
}
```

The key inequality is evaluated *before* a potentially overflowing multiplication. The output is unchanged on failure. This version accepts no leading sign or whitespace, by design.

C.3 Iterative factorial with an overflow check

```
#include <stdbool.h>
#include <limits.h>

bool factorial(unsigned n, unsigned long *out)
{
    if (out == nullptr) return false;
    unsigned long result = 1;
    unsigned i = 2;
    while (i <= n) {
        if (result > ULONG_MAX / i) return false;
        result *= i;
        if (i == n) break;
        ++i;
    }
    *out = result;
```

```
    return true;
}
```

The explicit final-iteration break also avoids unsigned loop-counter wraparound if `n == UINT_MAX`. Checking multiplication alone is not the whole correctness proof.

C.4 Little-endian byte decoding

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

bool read_u32_le(const unsigned char *bytes, size_t count, uint32_t *out)
{
    if (bytes == nullptr || out == nullptr || count < 4) return false;
    uint32_t result = (uint32_t)bytes[0]
                    | ((uint32_t)bytes[1] << 8)
                    | ((uint32_t)bytes[2] << 16)
                    | ((uint32_t)bytes[3] << 24);
    *out = result;
    return true;
}
```

The cast happens before shifting, so promotions do not make the top byte shift a signed overflow. Endianness is described by the wire format, not assumed from host memory.

C.5 Cleanup on every path

```
#include <stdio.h>
#include <stdbool.h>

bool write_record(const char *path)
{
    FILE *fp = fopen(path, "w");
    if (fp == nullptr) return false;
    bool ok = fputs("record\n", fp) != EOF;
    if (fclose(fp) == EOF) ok = false;
    return ok;
}
```

Notice that `fclose` runs even after `fputs` fails. The function reports stream-level success, not guaranteed durable storage on physical media.